ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

BÀI 10. Thiết kế lưu trữ và CSDL

ONE LOVE. ONE FUTURE.

Mục tiêu

- Hiểu vai trò của lưu trữ bền vững (object persistence) và cơ sở dữ liệu trong phân tích và thiết kế hệ thống thông tin hướng đối tượng.
- Biết cách chuyển đổi mô hình hướng đối tượng (UML Class Diagram) sang mô hình dữ liệu quan hệ (Relational Schema).
- Nắm được các bước thiết kế cơ sở dữ liệu:
  - từ mô hình khái niệm (conceptual),
  - qua mô hình logic (logical),
  - đến mô hình vật lý (physical).
- Áp dụng các quy tắc chuẩn hóa (normalization)
  - để tạo ra một thiết kế cơ sở dữ liệu hiệu quả, linh hoạt và tránh các dị thường dữ liệu (anomalies)
- Thực hành các kỹ năng cơ bản để thiết kế CSDL cho một bài toán quản lý quen thuộc.

Lưu trữ Bền vững

- Hầu hết các ứng dụng phần mềm đều vô dụng nếu không có dữ liệu.
- Dữ liệu cần tồn tại ngay cả khi ứng dụng đã tắt.
- Quá trình này gọi là lưu trữ bền vững (object persistence).

- object persistence Sự tồn tại của đối tượng
  - đề cập đến khả năng lưu trữ trạng thái của một đối tượng vượt quá thời gian tồn tại của tiến trình tạo ra nó, cho phép truy xuất và sử dụng lại sau này

- Khi quyết định lưu trữ dữ liệu,
  - chúng ta có nhiều lựa chọn, mỗi loại có ưu và nhược điểm riêng.
- Việc lựa chọn định dạng phù hợp là một quyết định kiến trúc quan trọng, ảnh hưởng đến toàn bộ hệ thống sau này.

Các định dạng Lưu trữ đối tượng (Object Persistence Formats)

- 1. Hệ thống dựa trên tệp (File-based Systems):
  - Đơn giản nhưng dễ gây dư thừa và dị thường dữ liệu (update anomalies).
- 2. Cơ sở dữ liệu Quan hệ (Relational Databases - RDBMS):
  - Phổ biến nhất hiện nay. Dữ liệu được tổ chức thành các bảng liên kết với nhau.
  - Ví dụ: Oracle, SQL Server, MySQL.
- 3. Cơ sở dữ liệu Hướng đối tượng (Object-Oriented Databases - OODBMS):
  - Hỗ trợ trực tiếp các khái niệm OO như kế thừa.
- 4. NoSQL Data Stores:
  - Phù hợp cho dữ liệu lớn, phân tán nhưng thường không đảm bảo tính nhất quán ngay lập tức (eventual consistency)

Tại sao RDBMS vẫn là lựa chọn hàng đầu?

- Dù có nhiều công nghệ CSDL các hệ quản trị CSDL quan hệ (RDBMS) vẫn thống trị trong các ứng dụng kinh doanh?
  - sự trưởng thành, ổn định và các tính năng mạnh mẽ mà chúng mang lại.
- Ưu điểm
  - Hỗ trợ toàn diện cho Client/Server:
    - Xử lý được phân tán trong toàn tổ chức, RDBMS cung cấp sức mạnh và sự linh hoạt cần thiết.
  - Kinh tế theo quy mô (Economy of scale):
    - Cho phép sử dụng phần cứng tốt hơn, xử lý khối lượng lớn với chi phí thấp hơn trên các máy chủ mạnh.
  - Bảo mật tốt hơn:
    - Cho phép định nghĩa các thủ tục ủy quyền để đảm bảo chỉ người dùng hợp pháp mới có thể truy cập CSDL.
  - Tăng năng suất lập trình viên:
    - Lập trình viên không cần tạo cấu trúc tệp cơ bản, cho phép họ tập trung vào thiết kế logic.

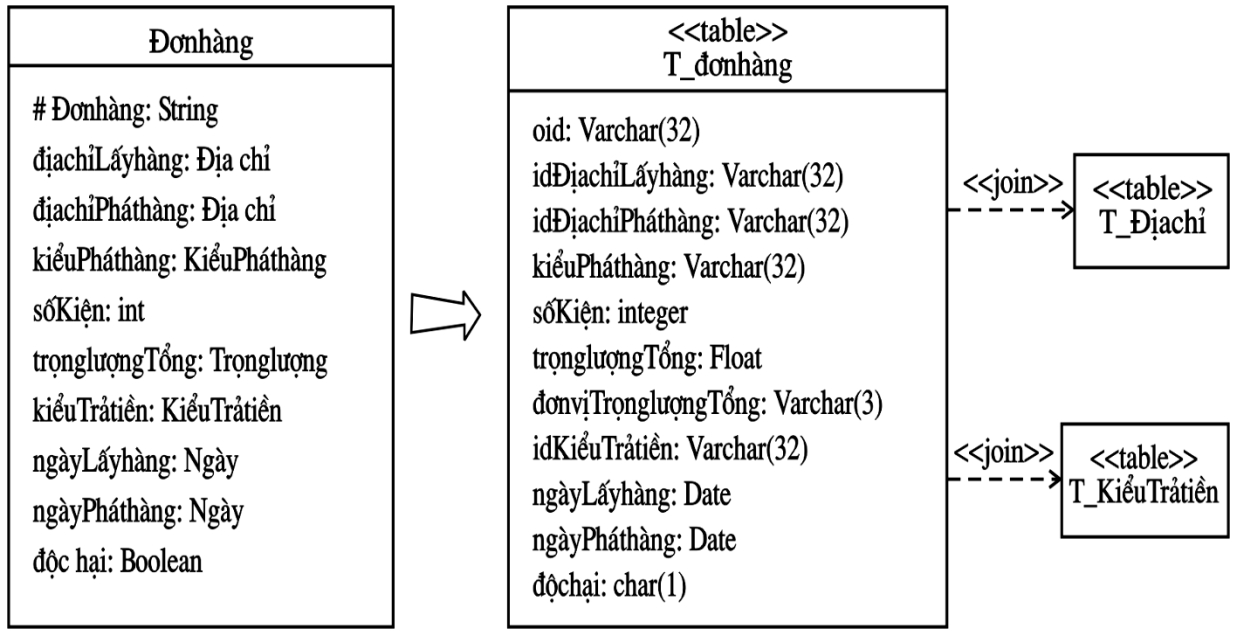
Thách thức: Sự không tương thích giữa Hướng đối tượng và Quan hệ

- Mặc dù RDBMS rất mạnh mẽ, nhưng nó lại được xây dựng trên một mô hình (quan hệ) khác biệt với mô hình trong phân tích (hướng đối tượng).
- Sự khác biệt này tạo ra một “Khác biệt" kỹ thuật mà chúng ta phải vượt qua

- Khác biệt
  - Phân tích Hướng đối tượng (OOA) tạo ra các mô hình lớp (class models).
  - Thiết kế CSDL Quan hệ (RDB) tạo ra các mô hình bảng (table models).
  - Sự khác biệt giữa hai mô hình này được gọi là "Trở kháng không tương thích" (Impedance Mismatch).
- Nhiệm vụ của chúng ta:
  - Xây dựng một "cây cầu" - một bộ quy tắc nhất quán - để chuyển đổi (ánh xạ) từ mô hình lớp sang mô hình bảng.

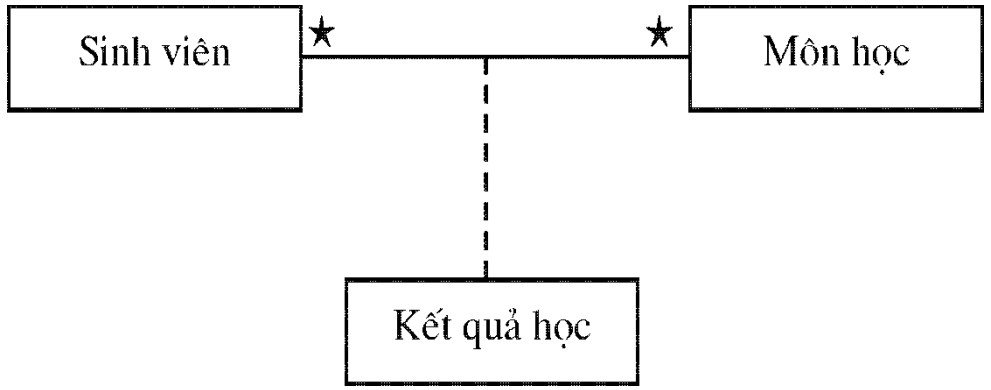
Công cụ trợ giúp Thiết kế lưu trữ dữ liệu

- Công cụ trợ giúp
  - sẽ biến đổi biểu đồ này thành ngôn ngữ định nghĩa dữ liệu (DDL),
  - cho phép tạo lập các bảng trong CSDL.
  - Vd: Rational Rose
- Ví dụ:
  - Ánh xạ lớp "Đơnhàng" sang bảng.
  - Kết quả của sự ánh xạ
    - một biểu đồ lớp (trong UML), được diễn tả theo cấu trúc lưu trữ,
    - mỗi lớp đều mang khuôn dập <<table>>, và tên lớp được gắn thêm tiền tố T_.



Vấn đề sử dụng công cụ

- Thiếu các bảng:
  - Như ta đã thấy trong mục thiết kế các liên kết, nếu giữa hai lớp có một liên kết nhiều-nhiều mà liên kết đó lại có thuộc tính,
    - thì phải định nghĩa một lớp liên kết chứa thuộc tính đó.
- Vd
  - giữa Sinh viên và Môn học có một liên kết nhiều-nhiều ("Sinh viên hoàn thành môn học");
  - liên kết này lại có
    - thuộc tính (như là năm học, kết quả thi),
    - do đó trong mô hình đối tượng, phải thêm một lớp liên kết là lớp “Kết quả học”.



- Chuyển sang mô hình quan hệ,
  - ngoài hai bảng Sinh viên và Môn học,
    - ta cũng có bảng Kết quả học.

Data Modeling

Các mức thiết kế Cơ sở dữ liệu

- Thiết kế CSDL tuân theo một quy trình tinh chỉnh dần dần qua ba mức độ trừu tượng, giúp tập trung vào đúng vấn đề ở từng giai đoạn.
  - 1. Mức Khái niệm (Conceptual):
    - Mô tả "sự vật" trong miền vấn đề mà không phụ thuộc công nghệ.
    - Công cụ: UML Class Diagram (OOAD) hoặc ERD (truyền thống).
  - 2. Mức Logic (Logical):
    - Chuyển đổi mô hình khái niệm thành một lược đồ quan hệ, độc lập với một hệ quản trị CSDL (DBMS) cụ thể.
    - Công việc: Xác định bảng, cột, khóa và chuẩn hóa (normalization).
  - 3. Mức Vật lý (Physical):
    - Mô tả cách dữ liệu sẽ được lưu trữ trên một DBMS cụ thể.
    - Công việc: Chọn kiểu dữ liệu, tối ưu hóa qua chỉ mục (indexing), tối ưu lưu trữ.

Mức 1 - Thiết kế Khái niệm (Conceptual Design)

- Conceptual Design
  - Là mức độ đầu tiên và trừu tượng nhất, ở đó cần một bộ "từ vựng" chung để trao đổi với người dùng về các khái niệm kinh doanh cốt lõi.
- Trong OOAD, công cụ mạnh mẽ nhất cho việc này chính là Sơ đồ Lớp Miền vấn đề.
  - Nó không chỉ giống với Sơ đồ Quan hệ Thực thể (ERD) truyền thống mà còn phong phú hơn nhờ hỗ trợ các khái niệm hướng đối tượng.
  - Mục tiêu:
    - Xây dựng một mô hình logic, giàu ngữ nghĩa về các thực thể dữ liệu, thuộc tính và mối quan hệ tự nhiên giữa chúng.

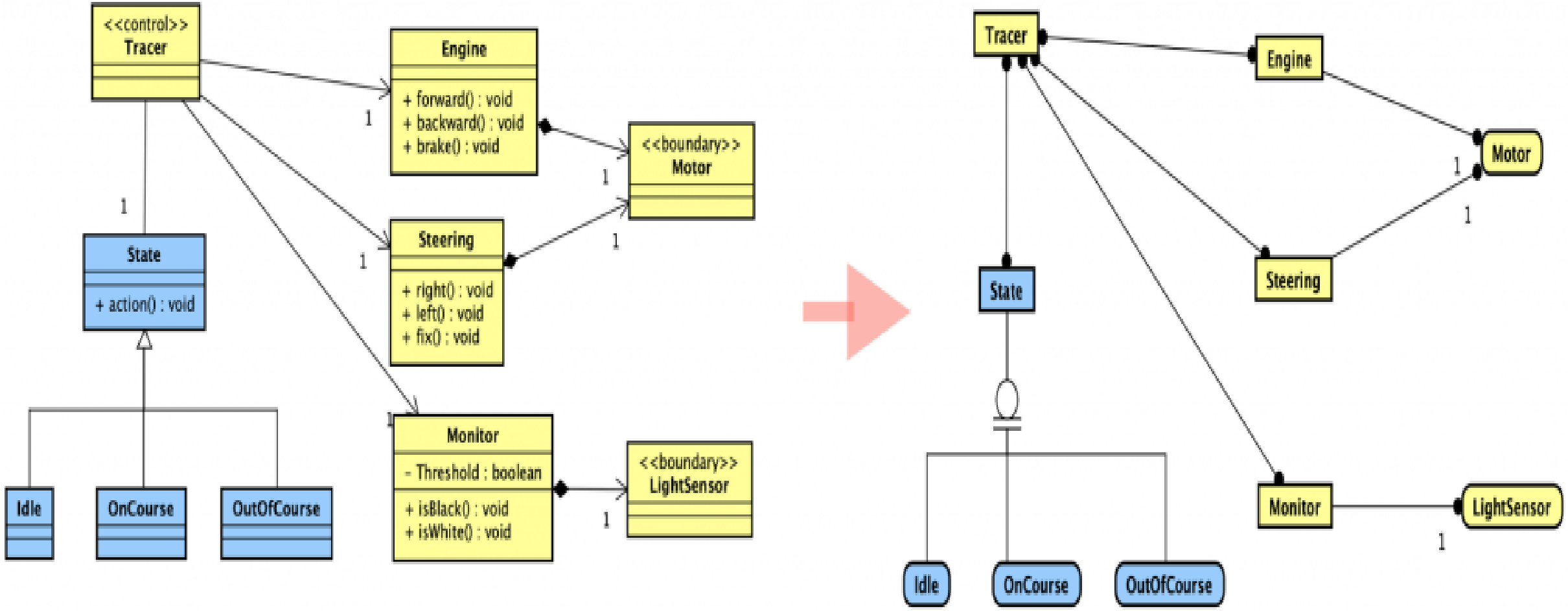
Công cụ chính trong OOAD

- Sơ đồ Lớp Miền vấn đề (Domain Model Class Diagram):
  - Đây là công cụ chính để mô hình hóa khái niệm.
- Công cụ chính: UML Domain Model Class Diagram
  - Lớp (Class):
    - Đại diện cho các "sự vật" hoặc khái niệm trong miền vấn đề
    - Ứng cử viên cho một Bảng (Table).
    - (ví dụ: Customer, Order, Product).
  - Thuộc tính (Attribute):
    - Các đặc điểm mô tả của mỗi lớp.
    - Ứng cử viên cho một Cột (Column).
  - Mối quan hệ (Association) và Bản số (Multiplicity):
    - Quyết định cách các bảng liên kết với nhau.
    - Các liên kết logic giữa các lớp
    - (ví dụ: Customer places Order).

So sánh với ERD

- Rất giống nhau ở mức khái niệm,
  - Cả hai đều mô tả các thực thể và mối quan hệ giữa chúng.
- Sự khác biệt chính:
  - Mục tiêu
    - ERD tập trung vào mô hình dữ liệu để triển khai cơ sở dữ liệu,
    - lớp miền tập trung vào mô hình đối tượng để triển khai phần mềm.
  - Hành vi:
    - ERD hoàn toàn mang tính cấu trúc.
    - Các lớp miền bao gồm rõ ràng các phương thức/hoạt động,
  - Bản đồ:
    - Các thực thể trong ERD thường ánh xạ trực tiếp tới các bảng,
    - một lớp miền duy nhất có thể được biểu diễn bằng nhiều bảng hoặc nhiều lớp miền có thể ánh xạ tới một bảng duy nhất, tùy thuộc vào chiến lược ánh xạ

Ví dụ Mức Khái niệm (Conceptual):



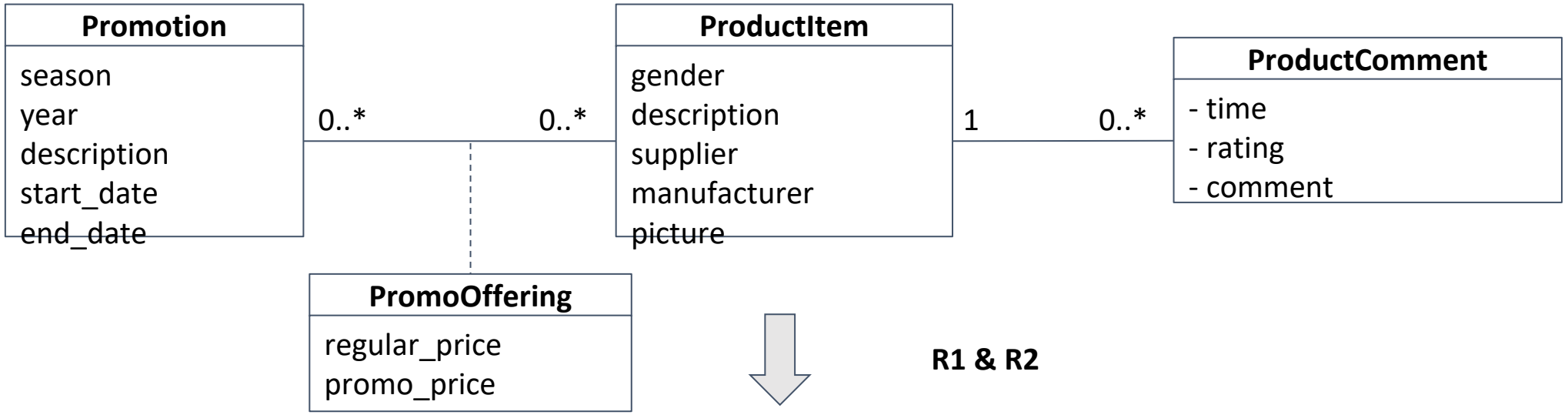
Mức 2 - Thiết kế Logic (Logical Design)

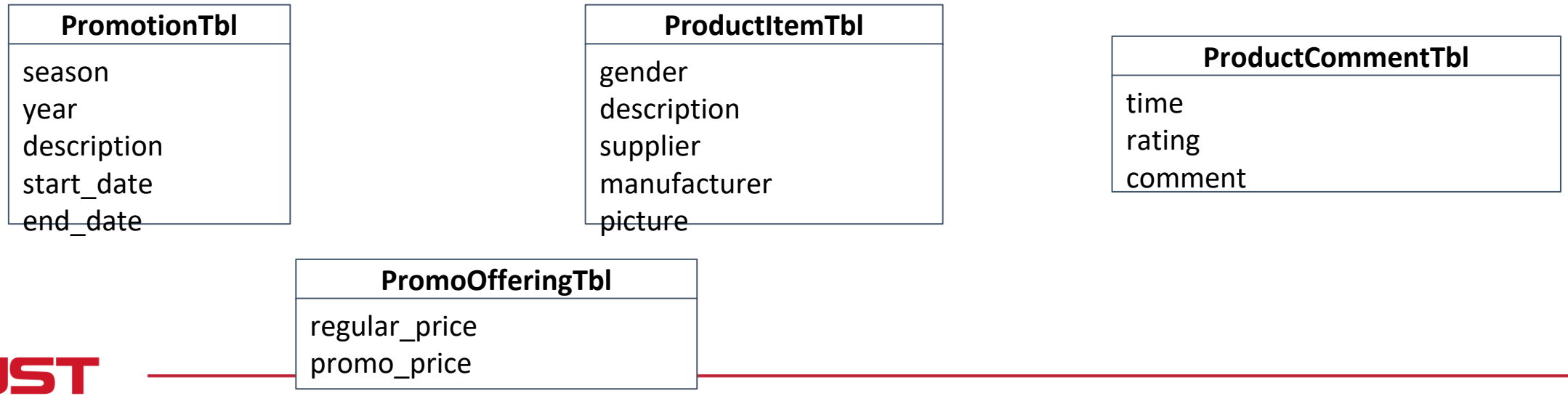
- Thiết kế logic
  - là quá trình chuyển đổi mô hình dữ liệu khái niệm thành một biểu diễn chi tiết, không phụ thuộc vào công nghệ của cấu trúc cơ sở dữ liệu.
  - Giai tập trung vào việc hiểu các yêu cầu nghiệp vụ và xác định các thực thể cùng mối quan hệ của chúng ở cấp độ cao.
- Thiết kế logic trong OOAD
  - Là “chuyển đổi” mô hình hướng đối tượng trừu tượng thành mô hình quan hệ có cấu trúc,
  - chuẩn bị cho việc triển khai vật lý, đảm bảo tính toàn vẹn và hiệu quả của dữ liệu
- Kỹ thuật để vượt qua "Trở kháng không tương thích" (Impedance Mismatch)
  - Bộ quy tắc nhất quán để “chuyển – ánh xạ" Sơ đồ lớp thành Quan hệ

Các bước chuyển

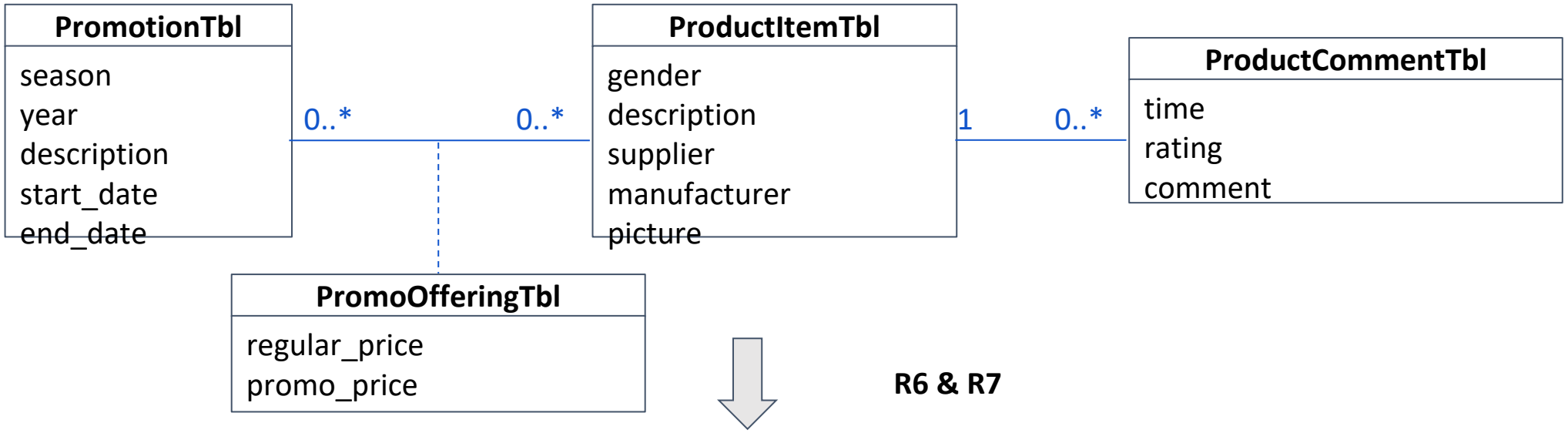
1. Chuyển đổi lớp thành bảng:
   - Mỗi lớp trong mô hình hướng đối tượng sẽ được ánh xạ thành một bảng trong cơ sở dữ liệu quan hệ.
2. Chuyển đổi thuộc tính thành cột:
   - Các thuộc tính của lớp sẽ trở thành các cột tương ứng trong bảng.
3. Ánh xạ định danh đối tượng thành khóa chính:
   - Định danh duy nhất của mỗi đối tượng (Object Identity) sẽ được sử dụng để tạo ra khóa chính (Primary Key) trong bảng của đối tượng đó.
4. Chuyển đổi mối quan hệ thành khóa ngoại:
   - Các mối quan hệ giữa các lớp, như Association, được biểu diễn bằng các khóa ngoại (Foreign Key)

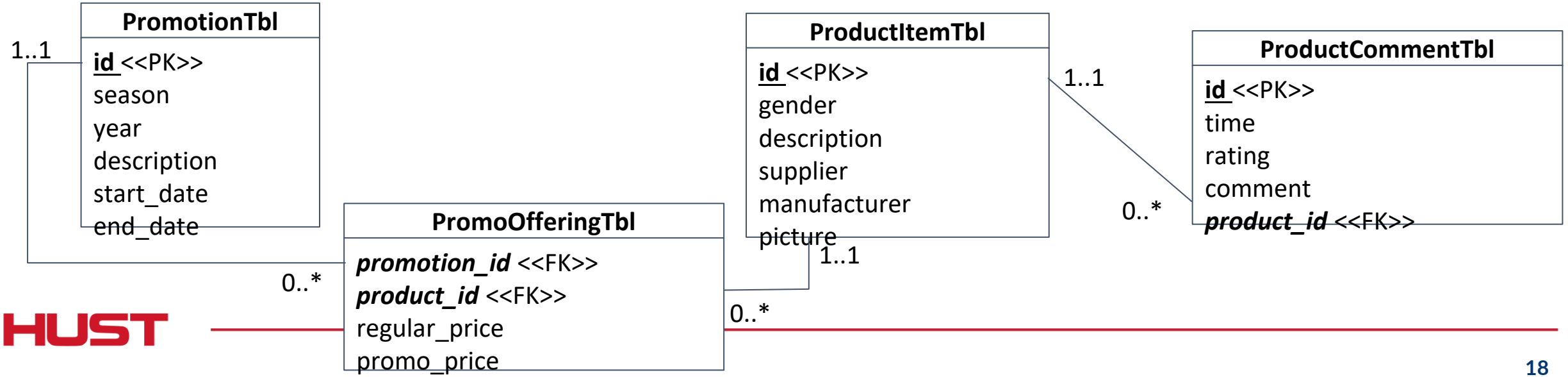
Mức Logic (Logical): Ánh xạ sang bảng





Xác định khóa và chuẩn hóa





Xử lý Quan hệ Association

- Cách xử lý (multiplicity)
- Một-một (1:1) :
  - Khóa ngoại cũng là khóa chính hoặc có ràng buộc duy nhất để thực thi mối quan hệ một-một.
- Một-nhiều (1:N) :
  - Một khóa ngoại được thêm vào bảng ở phía "nhiều" của mối quan hệ.
- Nhiều-nhiều (M:N) :
  - Một bảng liên kết (hoặc bảng nối) mới được tạo để lưu trữ mối quan hệ.

Xử lý Kế thừa (Inheritance):

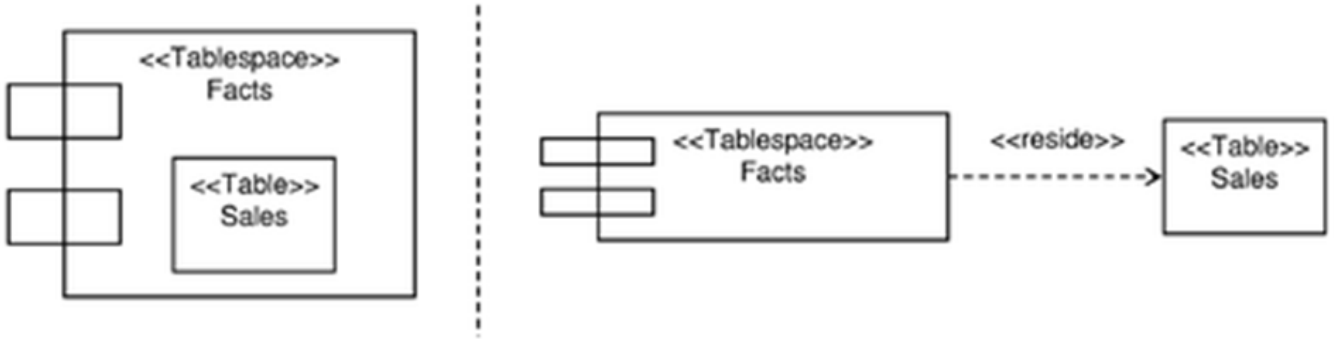
- Trong việc thiết kế cơ sở dữ liệu quan hệ khi chuyển đổi từ hệ thống đối tượng (OO) sang CSDL quan hệ, ba chiến lược phổ biến để xử lý kế thừa (inheritance) là
- Table-per-hierarchy:
  - Tạo một bảng duy nhất cho toàn bộ phân cấp, với các cột là hợp của tất cả các lớp con và lớp cha.
- Table-per-subclass:
  - Mỗi lớp (bao gồm cả lớp con) đều có một bảng riêng.
- Table-per-concrete-class:
  - Chỉ tạo bảng cho các lớp cụ thể (concrete classes), bỏ qua các lớp trừu tượng.

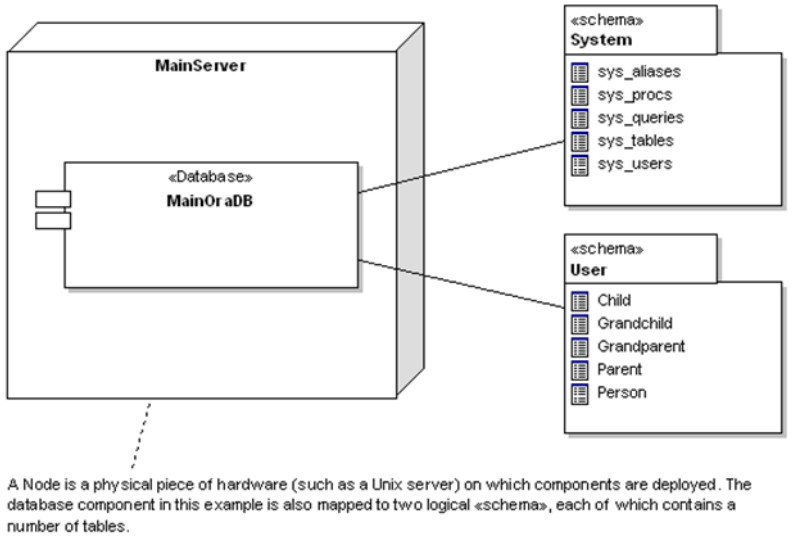
3. Mức 3 - Thiết kế Vật lý (Physical Design)

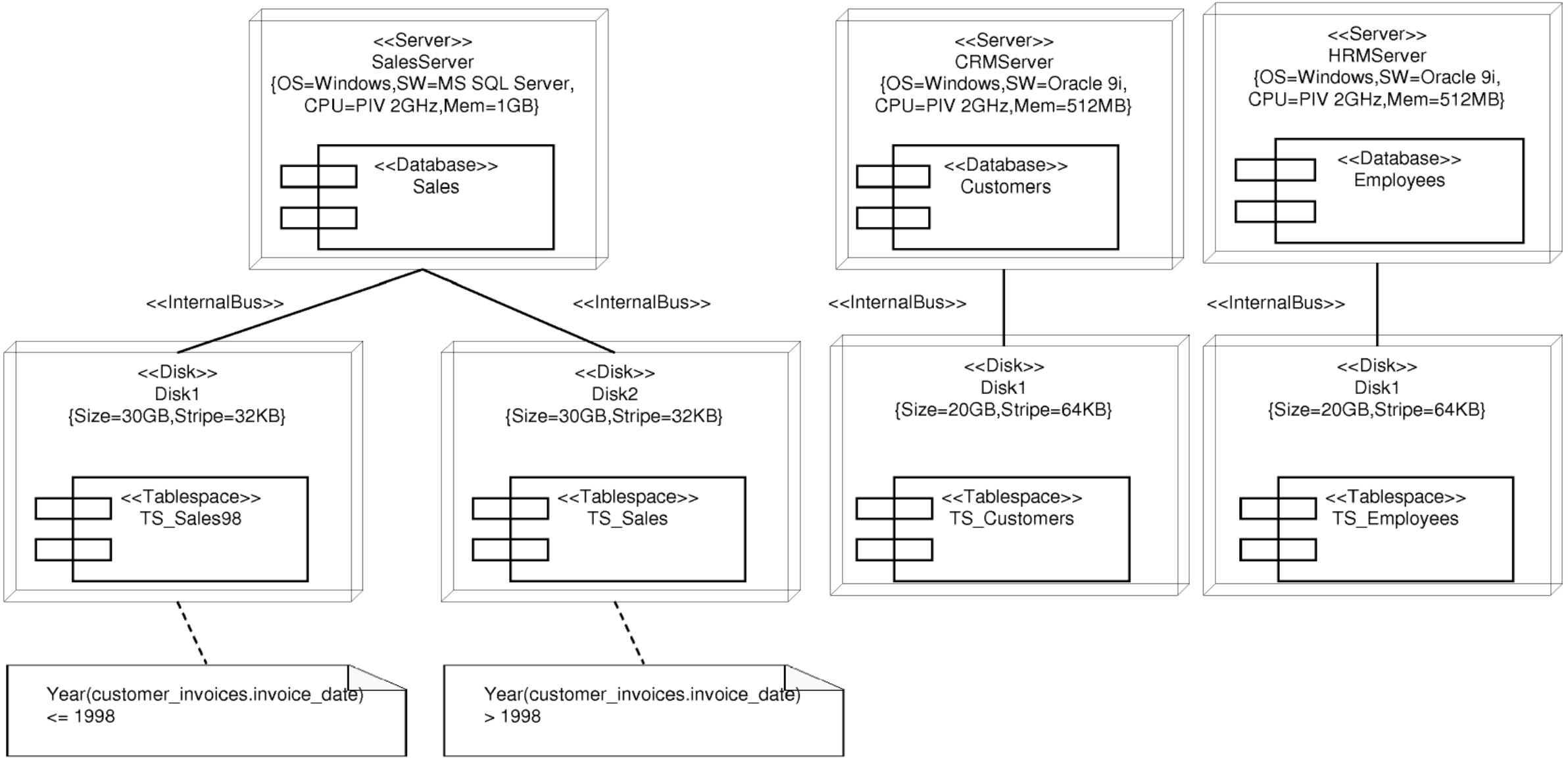
- Thiết kế cơ sở dữ liệu vật lý là quá trình chuyển đổi mô hình dữ liệu logic thành cấu trúc dữ liệu vật lý thực tế của Hệ quản trị cơ sở dữ liệu (DBMS)
  - (ví dụ: Oracle , MySQL, SQL Server)
- Mục tiêu:
  - tập trung vào cách dữ liệu sẽ được sắp xếp, lưu trữ và truy cập vật lý trên các thiết bị lưu trữ
  - Tối ưu hóa cho một DBMS cụ thể,
    - hiệu suất (performance) và
    - hiệu quả lưu trữ (storage efficiency).

Biểu đồ triển khai kiến trúc vật lý CSDL

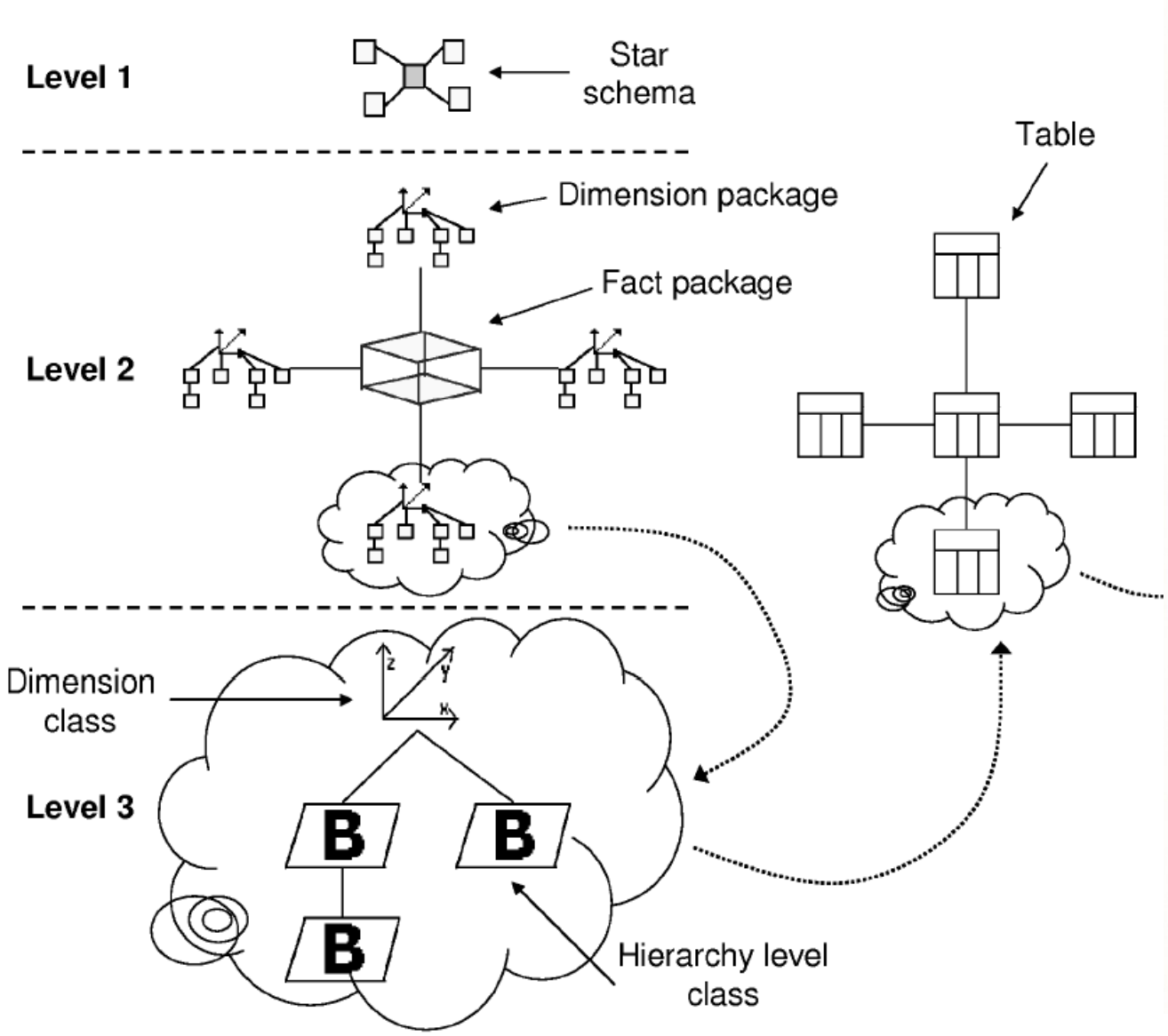
- Quá trình thiết kế cơ sở dữ liệu lấy các cấu trúc dữ liệu này và chuyển chúng thành lược đồ cơ sở dữ liệu vật lý,
  - sau đó được triển khai trên phần cứng cụ thể như một phần của quá trình triển khai vật lý.
- cho thấy cách các thành phần phần mềm
  - (tệp thực thi, thư viện, cơ sở dữ liệu)
- được triển khai trên các nút phần cứng
  - (máy chủ, thiết bị).

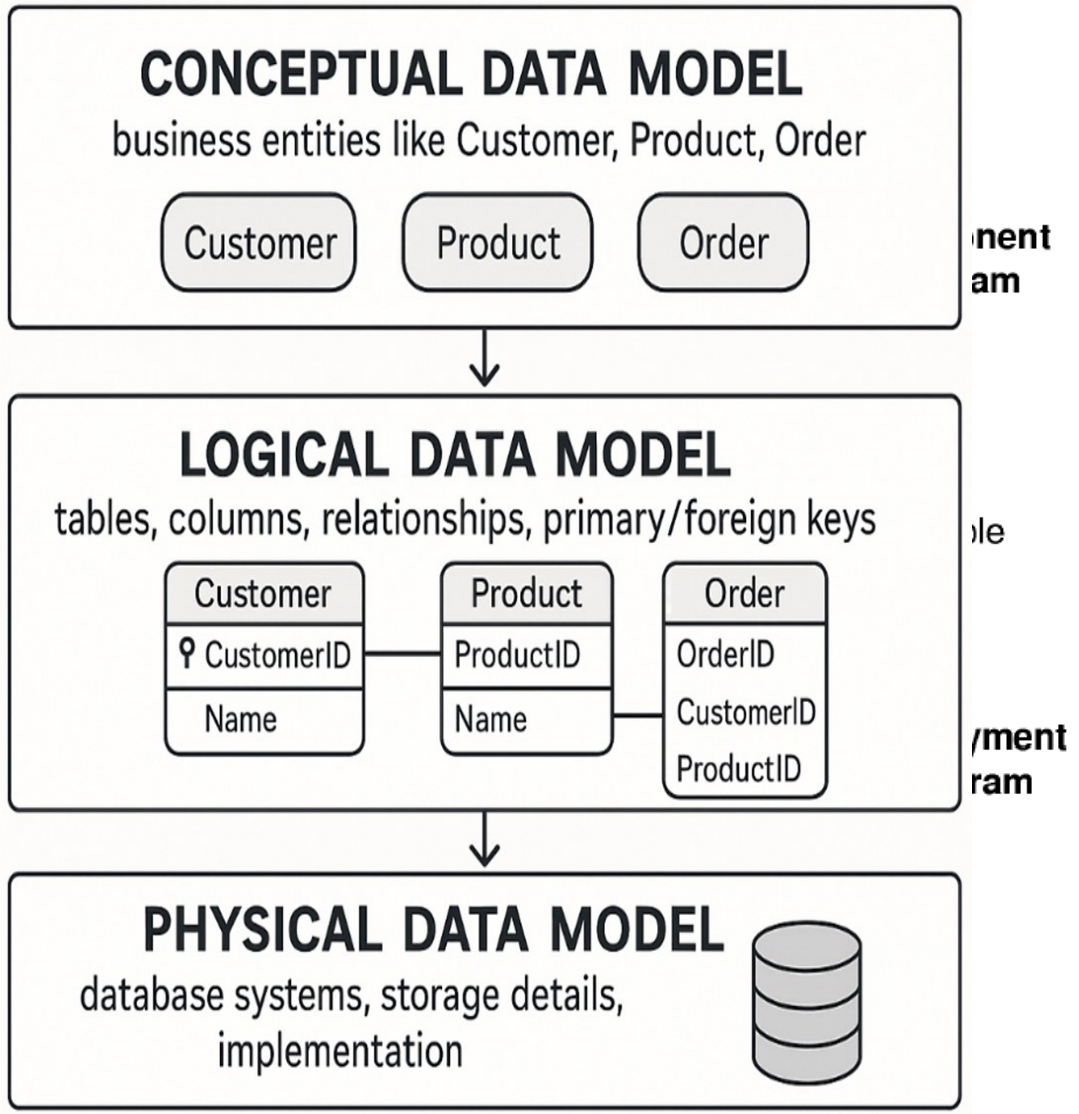





<<Server>>
SalesServer
{OS=Windows,SW=MS SQL Server,
CPU=PIV 2GHz,Mem=1GB}
<<Database>>
Sales
<<InternalBus>>
<<InternalBus>>
<<Disk>>
Disk1
{Size=30GB,Stripe=32KB}
<<Tablespace>>
TS_Sales98
<<Disk>>
Disk2
{Size=30GB,Stripe=32KB}
<<Tablespace>>
TS_Sales
Year(customer_invoices.invoice_date)
<= 1998
Year(customer_invoices.invoice_date)
> 1998
<<Server>>
CRMServer
{OS=Windows,SW=Oracle 9i,
CPU=PIV 2GHz,Mem=512MB}
<<Database>>
Customers
<<InternalBus>>
<<Disk>>
Disk1
{Size=20GB,Stripe=64KB}
<<Tablespace>>
TS_Customers
<<Server>>
HRMServer
{OS=Windows,SW=Oracle 9i,
CPU=PIV 2GHz,Mem=512MB}
<<Database>>
Employees
<<InternalBus>>
<<Disk>>
Disk1
{Size=20GB,Stripe=64KB}
<<Tablespace>>
TS_Employees

Tóm tắt các bước





Chuẩn hóa Dữ liệu
Data Normalization

Mối liên hệ giữa OOAD và Normalization

- Trong OOAD Khi chuyển sang thiết kế cơ sở dữ liệu, :
  - Chuyển đổi class thành bảng (table).
  - Thuộc tính trong class sẽ trở thành cột trong bảng.
  - Các mối quan hệ giữa các đối tượng sẽ được chuyển thành các khóa ngoại (foreign keys).
    - (ví dụ: association, aggregation)
- Việc ánh xạ từ sơ đồ lớp có thể tạo ra các bảng, nhưng chưa chắc các bảng đó đã có cấu trúc tốt.
  - Nếu không chuẩn hóa ?
    - Dữ liệu chưa được chuẩn hóa giống như một tủ quần áo không săp xếp
    - lộn xộn, khó tìm kiếm, dễ gây nhầm lẫn và tốn diện tích.
  - Cần có quá trình sắp xếp gọi là chuẩn hóa (Normalization)

Ví dụ. Chuẩn hóa

Dữ liệu phi chuẩn

Order
-Order Number : unsigned long -Date : Date -Cust ID : unsigned long -Last Name : String -First Name : String -State : String -Tax Rate : float -Product 1 Number : unsigned long -Product 1 Desc. : String -Product 1 Price : double -Product 1 Qty. : unsigned long -Product 2 Number : unsigned long -Product 2 Desc. : String -Product 2 Price : double -Product 2 Qty. : unsigned long -Product 3 Number : unsigned long -Product 3 Desc. : String -Product 3 Price : double -Product 3 Qty. : unsigned long

Redundant Data

Null Cells

Sample Records:

Order Number	Date	Cust ID	Last Name	First Name	State	Tax Rate	Prod. 1 Number	Prod. 1 Desc.	Prod. 1 Price	Prod. 1 Qty.	Prod. 2 Number	Prod. 2 Desc.	Prod. 2 Price	Prod. 2 Qty.
239	11/23/00	1035	Black	John	MD	0.05	555	Cheese Tray	$45.00	2				
260	11/24/00	1035	Black	John	MD	0.05	444	Wine Gift Pack	$60.00	1				
273	11/27/00	1035	Black	John	MD	0.05	222	Bottle Opener	$12.00	1				
241	11/23/00	1123	Williams	Mary	CA	0.08	444	Wine Gift Pack	$60.00	2				
262	11/24/00	1123	Williams	Mary	CA	0.08	222	Bottle Opener	$12.00	2				
287	11/27/00	1123	Williams	Mary	CA	0.08	222	Bottle Opener	$12.00	2				
290	11/30/00	1123	Williams	Mary	CA	0.08	555	Cheese Tray	$45.00	3				
234	11/23/00	2242	DeBerry	Ann	DC	0.065	555	Cheese Tray	$45.00	2				
237	11/23/00	2242	DeBerry	Ann	DC	0.065	111	Wine Guide	$15.00	1	444	Wine Gift Pack	$60.00	1
238	11/23/00	2242	DeBerry	Ann	DC	0.065	444	Wine Gift Pack	$60.00	1				
245	11/24/00	2242	DeBerry	Ann	DC	0.065	222	Bottle Opener	$12.00	1				
250	11/24/00	2242	DeBerry	Ann	DC	0.065	222	Bottle Opener	$12.00	1				
252	11/24/00	2242	DeBerry	Ann	DC	0.065	222	Bottle Opener	$12.00	1	444	Wine Gift Pack	$60.00	2
253	11/24/00	2242	DeBerry	Ann	DC	0.065	222	Bottle Opener	$12.00	1	444	Wine Gift Pack	$60.00	1

Chuẩn hóa (Normalization):

- Chuẩn hóa(Normalization):
  - là quá trình "dọn dẹp" và sắp xếp lại cấu trúc dữ liệu để nó trở nên hiệu quả, đáng tin cậy và dễ quản lý hơn.
- Cần một quy trình để tạo ra các bảng có cấu trúc tốt, hiệu quả, không dư thừa dữ liệu và loại bỏ các dị thường.
- chuẩn hóa cơ sở dữ liệu giúp loại bỏ các vấn đề:
  - Trùng lặp dữ liệu (redundancy) và
  - lỗi cập nhật (update anomalies),
  - đồng thời cải thiện hiệu suất của cơ sở dữ liệu.
- Đảm bảo tính toàn vẹn dữ liệu và tối ưu hóa lưu trữ.
  - Dạng chuẩn (Normal Forms), phổ biến nhất là 1NF, 2NF, và 3NF.

Ví dụ Dữ liệu phi chuẩn và chuẩn

SSN	Name	Department	Salary	Dependents
111-22-3333	Mary Smith	Accounting	40,000	John, Alice, Dave
222-33-4444	Jose Pena	Marketing	50,000	---
333-44-5555	Frank Collins	Production	35,000	Jan, Julia

SSN	Name	Department	Salary
111-22-3333	Mary Smith	Accounting	40,000
222-33-4444	Jose Pena	Marketing	50,000
333-44-5555	Frank Collins	Production	35,000

RecordID	SSN	Dependent
1	111-22-3333	John
2	111-22-3333	Alice
3	111-22-3333	Dave
4	333-44-5555	Jan
5	333-44-5555	Julia

Normalization Class Diagram

- Chuẩn hóa lớp
  - là một kỹ thuật phân tích giúp đảm bảo mỗi lớp có một vai trò và trách nhiệm duy nhất.
- Chuẩn hóa như một Công cụ "Kiểm tra Sức khỏe" cho Sơ đồ Lớp (Class Diagram)
  - đảm bảo các lớp có tính gắn kết cao (high cohesion) và khớp nối lỏng lẻo (low coupling)
- Mô hình hóa lớp chưa được "chuẩn hóa"
  - tiềm ẩn những vấn đề nghiêm trọng về tính toàn vẹn,
  - gây dư thừa và tạo ra các "dị thường" (anomalies) khi hệ thống hoạt động
- Thay vì chỉ xem chuẩn hóa là một bước sau khi đã có thiết kế bảng,
  - có thể áp dụng ngay trên Sơ đồ Lớp từ giai đoạn phân tích.
  - kiểm tra và tinh chỉnh chất lượng cho mô hình hướng đối tượng

Dạng chuẩn Thứ nhất (1NF) trong OOAD

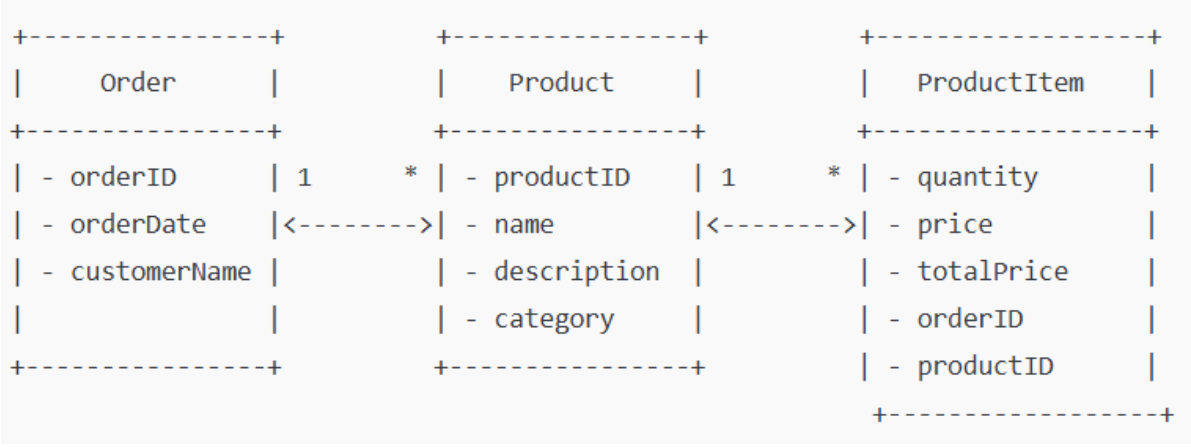
- Dạng chuẩn đầu tiên (1NF) yêu cầu chúng ta loại bỏ các "nhóm lặp".
  - Trong OOAD, nhóm lặp thường là dấu hiệu của một lớp bị bỏ sót hoặc một mối quan hệ nhiều-nhiều chưa được mô hình hóa đúng cách.
- Quy tắc 1NF:
  - Một lớp không được chứa các thuộc tính đa trị hoặc các nhóm thuộc tính lặp lại
- 1NF Check: Mỗi cột trong bảng phải có tên duy nhất
  - Lớp này có chứa các nhóm thuộc tính lặp lại không?
  - Nếu có, đây có phải là một lớp khác chưa được phát hiện?

- Ví dụ (Chưa đạt 1NF):
  - Lớp Order có các thuộc tính:
    - productID_1, productName_1, quantity_1
    - productID_2, productName_2, quantity_2
- Vấn đề:
  - Giới hạn số lượng sản phẩm trên mỗi đơn hàng.
  - Gây lãng phí không gian nếu đơn hàng chỉ có một sản phẩm.
  - Khó truy vấn: "Có bao nhiêu đơn hàng chứa sản phẩm X?".
- Tín hiệu thiết kế:
  - Nhóm thuộc tính lặp lại này cho thấy Product không phải là một thuộc tính của Order.
  - Nó là một lớp riêng biệt.

Giải pháp 1NF: Phát hiện Lớp và Lớp kết hợp

- Để đưa mô hình về dạng 1NF, chúng ta thực hiện một trong những kỹ thuật tái cấu trúc cơ bản nhất trong OOAD:
  - tách các nhóm thông tin lặp lại thành một lớp mới và tạo ra một lớp kết hợp để quản lý mối quan hệ.
- Hành động:
  - 1. Tạo lớp mới Product để chứa các thông tin về sản phẩm (productID, productName).
  - 2. Nhận ra mối quan hệ giữa Order và Product là nhiều-nhiều (M:N).
  - 3. Tạo một lớp kết hợp (association class) là OrderItem (hoặc OrderLine) để chứa các thuộc tính của mối quan hệ (quantity, price).

- Kết quả:
  - Lớp Order chỉ chứa thông tin về đơn hàng.
  - Lớp Product chỉ chứa thông tin về sản phẩm.
  - Lớp OrderItem chứa thông tin về một sản phẩm cụ thể trong một đơn hàng cụ thể.
- Lợi ích:
  - Mô hình trở nên linh hoạt, không giới hạn số lượng sản phẩm và loại bỏ dư thừa.



Ví dụ 2: một danh sách các “Sách" trong một đối tượng “Loan"),

- Phát hiện các Lớp còn thiếu (Identifying Missing Classes):
  - Chuẩn hóa giúp xác định các lớp mà chúng ta có thể đã bỏ qua trong quá trình phân tích ban đầu.
- Vấn đề trong OOAD:
  - khi chuyển sang cơ sở dữ liệu quan hệ,
  - ta sẽ phải chuẩn hóa (chia nhỏ bảng hoặc tách quan hệ)
  - để đảm bảo mỗi cột chỉ chứa một giá trị nguyên tử (1NF).

Vd Chuẩn hóa: Dạng chuẩn 1

Order 237 has 2 products

Sample Records:

Order Table

Order Number	Date	Cust ID	Last Name	First Name	State	Tax Rate
239	11/23/00	1035	Black	John	MD	0.05
260	11/24/00	1035	Black	John	MD	0.05
273	11/27/00	1035	Black	John	MD	0.05
241	11/23/00	1123	Williams	Mary	CA	0.08
262	11/24/00	1123	Williams	Mary	CA	0.08
287	11/27/00	1123	Williams	Mary	CA	0.08
290	11/30/00	1123	Williams	Mary	CA	0.08
234	11/23/00	2242	DeBerry	Ann	DC	0.065
237	11/23/00	2242	DeBerry	Ann	DC	0.065
238	11/23/00	2242	DeBerry	Ann	DC	0.065
245	11/24/00	2242	DeBerry	Ann	DC	0.065
250	11/24/00	2242	DeBerry	Ann	DC	0.065
252	11/24/00	2242	DeBerry	Ann	DC	0.065
253	11/24/00	2242	DeBerry	Ann	DC	0.065
297	11/30/00	2242	DeBerry	Ann	DC	0.065
243	11/24/00	4254	Bailey	Ryan	MD	0.05
246	11/24/00	4254	Bailey	Ryan	MD	0.05
248	11/24/00	4254	Bailey	Ryan	MD	0.05
235	11/23/00	9500	Chin	April	KS	0.05
242	11/23/00	9500	Chin	April	KS	0.05
244	11/24/00	9500	Chin	April	KS	0.05
251	11/24/00	9500	Chin	April	KS	0.05

Product Order Table

Order Number	Product Number	Product Desc	Product Price	Product Qty
239	555	Cheese Tray	$45.00	2
260	444	Wine Gift Pack	$60.00	1
273	222	Bottle Opener	$12.00	1
241	444	Wine Gift Pack	$60.00	2
262	222	Bottle Opener	$12.00	2
287	222	Bottle Opener	$12.00	2
290	555	Cheese Tray	$45.00	3
234	555	Cheese Tray	$45.00	2
237	111	Wine Guide	$15.00	1
237	444	Wine Gift Pack	$60.00	1
238	444	Wine Gift Pack	$60.00	1
245	222	Bottle Opener	$12.00	1
250	222	Bottle Opener	$12.00	1
252	222	Bottle Opener	$12.00	1
252	444	Wine Gift Pack	$60.00	2
253	222	Bottle Opener	$12.00	1
253	444	Wine Gift Pack	$60.00	1
297	333	Jams & Jellies	$20.00	2
243	555	Cheese Tray	$45.00	2
246	333	Jams & Jellies	$20.00	3
248	222	Bottle Opener	$12.00	1
248	333	Jams & Jellies	$20.00	2
248	111	Wine Guide	$15.00	1
235	222	Bottle Opener	$12.00	1
242	333	Jams & Jellies	$20.00	3
244	222	Bottle Opener	$12.00	2
251	111	Wine Guide	$15.00	2

Order 248 has 3 products

Áp dụng Nguyên tắc 1NF:

- Dạng chuẩn thứ nhất (1NF) yêu cầu loại bỏ các nhóm lặp.
  - Trong OOAD, điều này cho thấy Sách không phải là một thuộc tính đơn giản của "Mượn"- Loan
  - Nó nên là một lớp riêng biệt (Sách) và có mối quan hệ nhiều-nhiều (M:N) với lớp "Mượn".
  - Mối quan hệ này thường được hiện thực hóa thông qua một lớp kết hợp (association class), chẳng hạn như LoanItem hoặc LoanLine.
- Việc áp dụng 1NF giúp chúng ta phát hiện và tạo ra các lớp Sach và LoanItem còn thiếu,
  - làm cho mô hình đối tượng trở nên chính xác và linh hoạt hơn.

Dạng chuẩn Thứ hai (2NF) trong OOAD

- Dạng chuẩn thứ hai (2NF)
  - giúp chúng ta đảm bảo rằng tất cả các thuộc tính trong một lớp kết hợp (lớp có định danh phức hợp) đều phụ thuộc vào định danh đó.
  - Nếu một thuộc tính chỉ phụ thuộc vào một phần, nó đã bị đặt sai chỗ.
- Quy tắc 2NF:
  - Một lớp phải ở dạng 1NF và
  - mọi thuộc tính không phải định danh phải phụ thuộc hoàn toàn vào toàn bộ định danh (primary key).
- 2NF Check: Mọi thuộc tính không khóa phải phụ thuộc hoàn toàn vào khóa chính..
  - Nếu đây là lớp kết hợp, có thuộc tính nào chỉ mô tả một phần của định danh không?

Ví dụ Chuẩn hóa: Dạng chuẩn 2

Customer Table

Cust ID	Last Name	First Name
1035	Black	John
1123	Williams	Mary
2242	DeBerry	Ann
4254	Bailey	Ryan
9500	Chin	April

Last Name and First Name was moved to the Customer table to eliminate redundancy

Order Table

Order Number	Date	Cust ID	State	Tax Rate
239	11/23/00	1035	MD	0.05
260	11/24/00	1035	MD	0.05
273	11/27/00	1035	MD	0.05
241	11/23/00	1123	CA	0.08
262	11/24/00	1123	CA	0.08
287	11/27/00	1123	CA	0.08
290	11/30/00	1123	CA	0.08
234	11/23/00	2242	DC	0.065
237	11/23/00	2242	DC	0.065
238	11/23/00	2242	DC	0.065
245	11/24/00	2242	DC	0.065
250	11/24/00	2242	DC	0.065
252	11/24/00	2242	DC	0.065
253	11/24/00	2242	DC	0.065
297	11/30/00	2242	DC	0.065
243	11/24/00	4254	MD	0.05
246	11/24/00	4254	MD	0.05
248	11/24/00	4254	MD	0.05
235	11/23/00	9500	KS	0.05
242	11/23/00	9500	KS	0.05
244	11/24/00	9500	KS	0.05
251	11/24/00	9500	KS	0.05

Product Order Table

Order Number	Product Number	Product Qty
239	555	2
260	444	1
273	222	1
241	444	2
262	222	2
287	222	2
290	555	3
234	555	2
237	111	1
237	444	1
238	444	1
245	222	1
250	222	1
252	222	1
252	444	2
253	222	1
253	444	1
297	333	2
243	555	2
246	333	3
248	222	1
248	333	2
248	111	1
235	222	1
242	333	3
244	222	2
251	111	2

Product Table

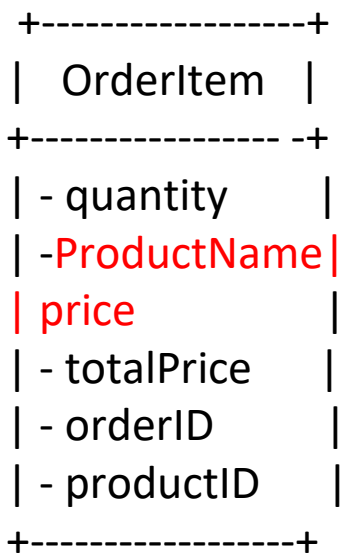
Product Number	Product Desc	Product Price
111	Wine Guide	$15.00
222	Bottle Opener	$12.00
333	Jams & Jellies	$20.00
444	Wine Gift Pack	$60.00
555	Cheese Tray	$45.00

Product Desc and Price was moved to the Product table to eliminate redundancy

tax & state?

Ví dụ (Chưa đạt 2NF):

- Giả sử lớp OrderItem có định danh là (orderID, productID) và các thuộc tính sau:
  - quantity
  - productName
- Phân tích phụ thuộc:
  - quantity:
  - Phụ thuộc vào cả orderID và productID (Số lượng của sản phẩm X trong đơn hàng Y) → OK.
- productName:
  - Chỉ phụ thuộc vào productID → VI PHẠM 2NF.
- Tín hiệu thiết kế:
  - productName không thuộc về trách nhiệm của OrderItem.
  - Nó thuộc về Product.



Giải pháp 2NF: Đặt thuộc tính đúng chỗ

- Giải pháp cho vi phạm 2NF rất đơn giản:
  - di chuyển thuộc tính bị phụ thuộc một phần về đúng lớp của nó.
  - Hành động này giúp tăng tính gắn kết (cohesion) cho cả hai lớp liên quan.
- Hành động:
  - Di chuyển thuộc tính productName từ lớp OrderItem sang lớp Product.
- Lớp OrderItem (đã đạt 2NF):
  - Định danh: (orderID, productID)
  - Thuộc tính: quantity
- Lớp Product (tăng tính gắn kết):
  - Định danh: productID
  - Thuộc tính: productName, price...
- Lợi ích:
  - productName chỉ được lưu trữ một lần cho mỗi sản phẩm.
  - Loại bỏ dị thường cập nhật:
  - Khi tên sản phẩm thay đổi, chỉ cần cập nhật ở một nơi duy nhất.

Cải thiện Tính gắn kết của Lớp (Improving Class Cohesion):

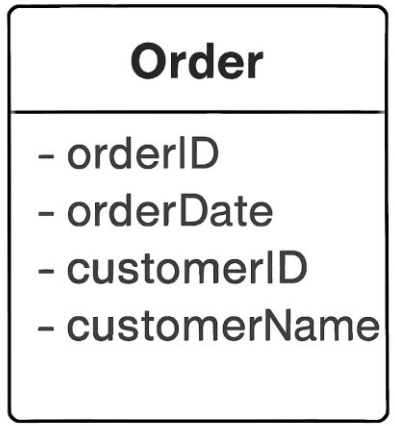
- Chuẩn hóa đảm bảo rằng mỗi lớp chỉ đại diện cho một khái niệm duy nhất.
- Vấn đề trong OOAD:
  - Một lớp có thể chứa các thuộc tính không thực sự mô tả thực thể chính của lớp đó.
  - Ví dụ, lớp OrderItem chứa các thuộc tính
    - productID,
    - productDescription,
    - quantity.
- Áp dụng Nguyên tắc 2NF:
  - productDescription chỉ phụ thuộc vào productID, không phụ thuộc vào orderID.
  - Điều này báo hiệu rằng
    - productDescription không thuộc về trách nhiệm của lớp OrderItem.
    - Nó thực sự là một thuộc tính của lớp Product.
- Việc tách các thuộc tính này ra một lớp riêng (lớp Product) giúp tăng tính gắn kết cho cả hai lớp OrderItem và Product.

Dạng chuẩn Thứ ba (3NF) trong OOAD

- Dạng chuẩn thứ ba (3NF) giúp chúng ta loại bỏ các "phụ thuộc bắc cầu".
  - Điều này có nghĩa là một thuộc tính không được phép phụ thuộc vào một thuộc tính khác không phải là định danh.
  - Đây là dấu hiệu rõ ràng nhất cho thấy một thuộc tính đang ở sai lớp.
- Quy tắc 3NF:
  - Một lớp phải ở dạng 2NF và
  - không có thuộc tính không phải định danh nào phụ thuộc bắc cầu vào định danh.
- 3NF Check: Các thuộc tính không khóa không phụ thuộc vào nhau.
  - Có thuộc tính nào trong lớp này mô tả một đối tượng khác hơn là đối tượng chính không?
  - (Ví dụ: customerName trong Order thực sự đang mô tả Customer).

Ví dụ (Chưa đạt 3NF):

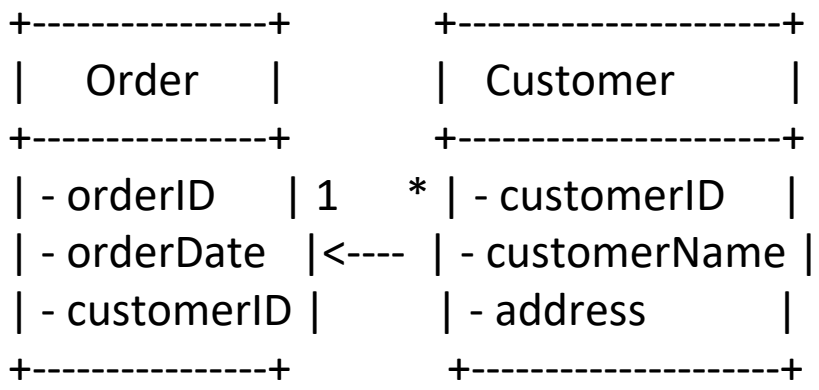
- Lớp Order có các thuộc tính:
  - orderID (Định danh)
  - orderDate
  - customerID
  - customerName



- Phân tích phụ thuộc:
  - orderDate phụ thuộc trực tiếp vào orderID → OK.
  - customerID phụ thuộc trực tiếp vào orderID → OK.
  - customerName phụ thuộc vào customerID, và customerID phụ thuộc vào orderID → PHỤ THUỘC BẮC CẦU → VI PHẠM 3NF.
- Tín hiệu thiết kế:
  - customerName không thuộc về trách nhiệm của Order.
  - Nó thuộc về Customer.

Giải pháp 3NF: Tách thành các lớp chuyên biệt

- giải pháp 3NF,
  - di chuyển thuộc tính phụ thuộc bắc cầu sang lớp mới .
  - đảm bảo mỗi lớp chỉ chứa thông tin mà nó chịu trách nhiệm.
- Hành động:
  - 1. Tạo lớp mới Customer.
  - 2. Di chuyển các thuộc tính customerID và customerName sang lớp Customer.
  - 3. Lớp Order giữ lại customerID như một tham chiếu (sẽ trở thành khóa ngoại khi ánh xạ CSDL).
- Lớp Order (đã đạt 3NF):
  - Định danh: orderID
  - Thuộc tính: orderDate, customerID (tham chiếu)
  - Lớp Customer (mới):
  - Định danh: customerID
  - Thuộc tính: customerName, address...
- Lợi ích:
  - Mỗi lớp có tính gắn kết cao, loại bỏ hoàn toàn dư thừa và các dị thường.

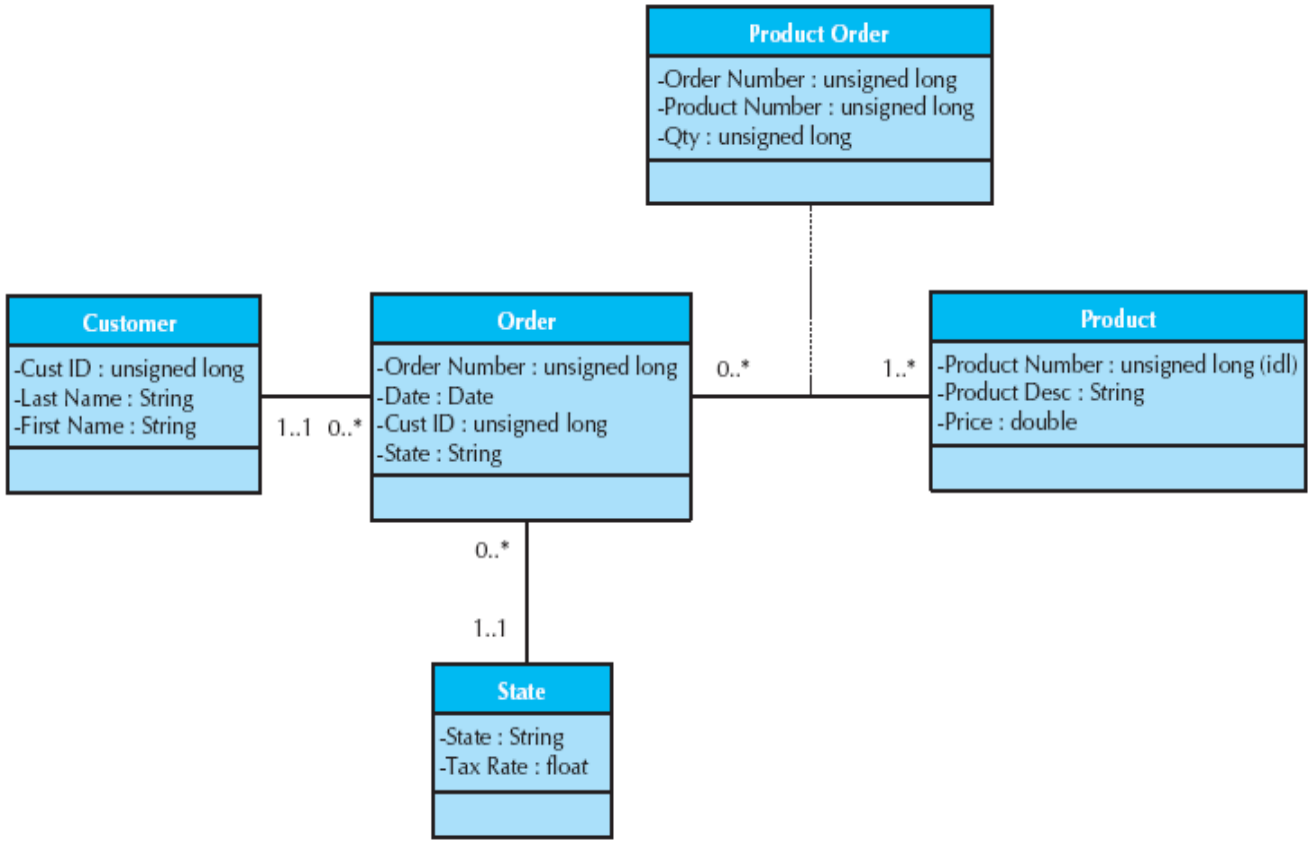


(Removing Unwanted Dependencies):

- Loại bỏ các Phụ thuộc không mong muốn
  - Vấn đề trong OOAD:
  - Một lớp có thể chứa các thuộc tính phụ thuộc vào các thuộc tính khác không phải là định danh (key) của lớp đó.
  - Ví dụ, lớp Order có thể chứa customerID và customerName.

- Dạng chuẩn thứ ba (3NF) yêu cầu loại bỏ các phụ thuộc bắc cầu (transitive dependencies).
  - Ở đây, customerName phụ thuộc vào customerID.
  - Điều này cho thấy customerName không thuộc về trách nhiệm của lớp Order mà thuộc về lớp Customer.
  - Việc áp dụng 3NF giúp chúng ta xác nhận rằng các thuộc tính đã được đặt đúng lớp, từ đó giảm khớp nối và sự dư thừa.

- Ví dụ 2:
  - Khi làm việc với các mối quan hệ trong OOAD (chẳng hạn như quan hệ 1-n hoặc n-n),
  - khi chuyển sang cơ sở dữ liệu quan hệ,
    - ta cần phải đảm bảo rằng các mối quan hệ này được biểu diễn đúng cách trong lược đồ cơ sở dữ liệu,
    - và chuẩn hóa sẽ giúp phân tách các bảng để tránh sự dư thừa hoặc ràng buộc không cần thiết.



Phi chuẩn (Denormalization)?

- Chuẩn hóa giúp chúng ta tạo ra một thiết kế logic lý tưởng.
  - Tuy nhiên, trong thực tế, đôi khi chúng ta cần phải "phá vỡ" quy tắc một chút để tối ưu hóa hiệu suất truy vấn.
  - Quá trình này được gọi là phi chuẩn hóa và phải được thực hiện một cách cẩn thận.
- Phi chuẩn hóa (Denormalization):
  - Là quá trình thêm dữ liệu dư thừa vào một bảng đã được chuẩn hóa để cải thiện hiệu suất đọc (giảm số lần JOIN).
- Khi nào cần xem xét?
  - Các báo cáo cần tổng hợp dữ liệu từ nhiều bảng và được chạy thường xuyên.
  - Các mối quan hệ 1-1 thường được truy cập cùng nhau.
- Ưu điểm:
  - Tăng tốc độ truy vấn.
- Nhược điểm:
  - Tăng độ phức tạp khi cập nhật, lãng phí không gian lưu trữ và có nguy cơ gây ra các dị thường.
- Kết luận: Chỉ thực hiện phi chuẩn hóa khi có lý do hiệu suất rõ ràng và đã được đo lường. Luôn bắt đầu với một thiết kế đã được chuẩn hóa.

Chuẩn hóa là Kỹ thuật Thiết kế, không chỉ là Quy tắc cho CSDL

- Trong bối cảnh OOAD, chuẩn hóa không nên bị giới hạn trong vai trò là một bộ quy tắc máy móc áp dụng cho các bảng CSDL.
- Thay vào đó, nó là một tư duy thiết kế giúp chúng ta:
  - Phân tích và xác nhận chất lượng của các lớp trong mô hình miền vấn đề.
  - Tinh chỉnh Sơ đồ Lớp để đảm bảo các lớp có trách nhiệm rõ ràng, chuyên biệt.
  - đưa ra quyết định tốt hơn khi ánh xạ cấu trúc kế thừa phức tạp trong OOAD sang mô hình CSDL quan hệ,.
- Bằng cách áp dụng tư duy chuẩn hóa từ sớm,
  - có thể tạo ra một mô hình hướng đối tượng không chỉ phản ánh đúng thế giới thực mà còn là một nền tảng vững chắc,
  - dễ dàng chuyển đổi thành một thiết kế cơ sở dữ liệu quan hệ hiệu quả và dễ bảo trì.

Quy tắc ánh xạ

Impedance Mismatch

- Các mô hình hướng đối tượng (OOAD)
  - Vd: Sơ đồ Lớp (Class Diagram),
  - mô tả thế giới thực một cách tự nhiên thông qua các đối tượng, thuộc tính và mối quan hệ.
  - Mô hình đối tượng vốn phong phú với kế thừa và các mối quan hệ phức tạp
- Khi chuyển sang giai đoạn thiết kế CSDL,
  - đặc biệt là CSDL quan hệ (RDBMS), định dạng lưu trữ phổ biến nhất sử dụng mô hình quan hệ
  - mô hình quan hệ Dữ liệu được lưu trữ trong các bảng phẳng đơn giản và các mối quan hệ được quản lý thông qua khóa ngoại.
- Tồn tại thách thức lớn
  - gọi là "Trở kháng không tương thích" (Impedance Mismatch).

Maping rules

- Sự không phù hợp này được khắc phục bằng một bộ quy tắc ánh xạ chuyển đổi các khái niệm OO thành các khái niệm quan hệ
- Maping rules
  - là bộ quy tắc hệ thống để chuyển đổi từng thành phần của Sơ đồ Lớp (Class Diagram) thành các thành phần tương ứng trong Lược đồ Quan hệ (Relational Schema) giúp chúng ta bắc cây cầu qua "hố ngăn cách" này.
- Object-Relational Mapping (ORM)
  - Quá trình này còn được gọi là Ánh xạ đối tượng-quan hệ
  - Thách thức chính mà ORM giải quyết là trở kháng quan hệ đối tượng",
    - sự khác biệt cơ bản giữa các chương trình hướng đối tượng và cơ sở dữ liệu quan hệ mô hình hóa thông tin.

Các quy tắc chuyển đổi (Mapping Rules)

- Quy tắc 1: Ánh xạ Lớp (Class)
  - Mỗi lớp cố định trong mô hình OOAD được ánh xạ vào một bảng trong cơ sở dữ liệu.
- Quy tắc 2: Ánh xạ Thuộc tính (Attribute)
  - Mỗi thuộc tính của một lớp được ánh xạ tới một cột trong bảng tương ứng.
- Quy tắc 3:
  - Ánh xạ Định danh Đối tượng (Object Identity)
- Quy tắc 4:
  - Ánh xạ Quan hệ (Association)
- Quy tắc 5:
  - Ánh xạ Kế thừa (Generalization)

Quy tắc 1 Class → Table,

- Mỗi khái niệm chính (Lớp) trong thế giới hướng đối tượng sẽ trở thành một "thùng chứa" dữ liệu (Bảng) trong thế giới quan hệ.
- Tương tự, mỗi đặc điểm mô tả (Thuộc tính) của lớp sẽ trở thành một cột thông tin trong bảng đó.
- Quy tắc 1:
  - Mỗi lớp cụ thể (concrete class) được ánh xạ thành một bảng (Table).
  - Ví dụ: Lớp CUSTOMER → Bảng CUSTOMER.

Quy tắc 2: Attribute → Column

- Mỗi thuộc tính đơn trị (single-valued attribute)
  - trở thành một cột (Column).
  - Ví dụ:
    - Thuộc tính customerName → Cột customerName.
- Các thuộc tính đa trị (multivalued attributes)
  - Ví dụ: Ví dụ bao gồm nhiều số điện thoại hoặc địa chỉ email của một người
  - Giải pháp:
    - Tạo một bảng mới cho thuộc tính đa giá trị.
    - Bảng mới này bao gồm một khóa ngoại liên kết trở lại với khóa chính của bảng thực thể ban đầu.
    - Mỗi hàng trong bảng mới chứa một giá trị duy nhất cho thuộc tính đa giá trị và khóa ngoại tương ứng.

Quy tắc 3: Object Identity

- Bản sắc của một đối tượng, đặc điểm vốn có trong chính đối tượng đó,
  - phải được ánh xạ rõ ràng tới khóa chính trong bảng cơ sở dữ liệu
- Trong thế giới thực, mỗi đối tượng là duy nhất.
  - Làm thế nào để đảm bảo tính duy nhất này trong cơ sở dữ liệu?
- Chúng ta sử dụng một khái niệm gọi là Khóa chính (Primary Key).
  - Đây là một cột (hoặc tổ hợp cột) có giá trị không bao giờ trùng lặp, giúp phân biệt từng hàng dữ liệu.
- Các loại khóa chính:
  - Khóa tự nhiên:
    - Khóa chính được lấy từ các thuộc tính hiện có trong dữ liệu vốn là duy nhất
  - Khóa thay thế:
    - Khóa nhân tạo, số tự động tăng hoặc GUID, được thêm vào để làm khóa chính.
  - Khóa chính tổng hợp:
    - Khóa chính bao gồm nhiều cột có giá trị kết hợp xác định duy nhất.

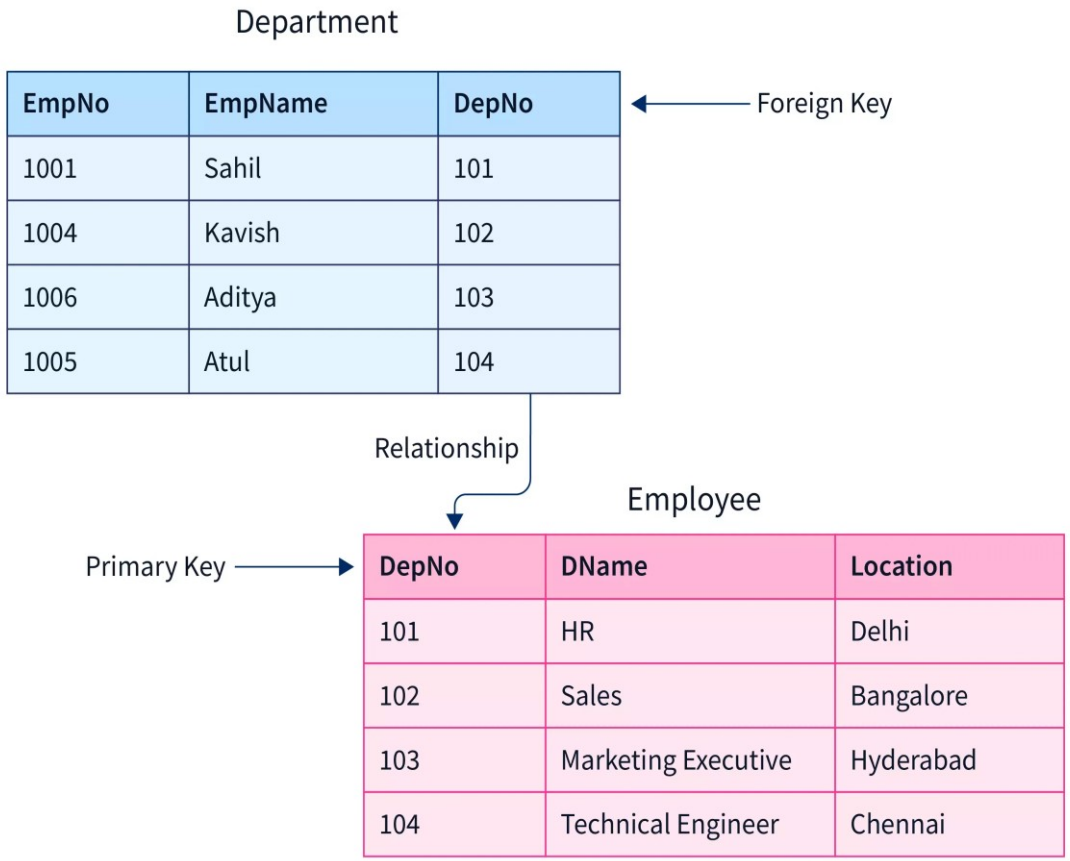
Các đặc điểm chính của Khóa chính

- Tính độc đáo:
  - Mỗi giá trị trong cột khóa chính phải là duy nhất trên tất cả các hàng trong bảng.
- Không thể Nullability:
  - Các cột khóa chính không thể chứa giá trị NULL.
  - Mỗi bản ghi phải có một khóa chính xác định và có thể nhận dạng được.
- Khóa chính duy nhất cho mỗi bảng:
  - Mỗi bảng chỉ có thể có một khóa chính được xác định.
- Tính toàn vẹn dữ liệu:
  - ngăn chặn các bản ghi trùng lặp và đảm bảo rằng mỗi hàng đều riêng biệt.
- Nền tảng mối quan hệ:
  - nền tảng để thiết lập mối quan hệ giữa các bảng thông qua khóa ngoại, tham chiếu đến khóa chính

Quy tắc 4: Ánh xạ quan hệ Association , Foreign Key

- Các đối tượng không tồn tại một mình; chúng có mối quan hệ với nhau.

- Khóa ngoại (Foreign Key).
  - CSDL quan hệ thể hiện các mối quan hệ này bằng khóa ngoại
  - Đây là một "tham chiếu" từ một bảng này đến khóa chính của một bảng khác,
  - tạo ra một liên kết logic giữa chúng.
- Khóa ngoại là một cột trong một bảng, có giá trị phải khớp với một giá trị khóa chính trong một bảng khác.



Quan hệ giữa Association → Foreign Key

- Cách xử lý (multiplicity)
- Một-một (1:1) :
  - Khóa ngoại cũng là khóa chính hoặc có ràng buộc duy nhất để thực thi mối quan hệ một-một.
- Một-nhiều (1:N) :
  - Một khóa ngoại được thêm vào bảng ở phía "nhiều" của mối quan hệ.
- Nhiều-nhiều (M:N) :
  - Một bảng liên kết (hoặc bảng nối) mới được tạo để lưu trữ mối quan hệ.

Mối quan hệ 1-1 (One-to-One):

- Trong mô hình OO:
  - Một đối tượng A có thể liên kết với một đối tượng B, và ngược lại.
  - Ví dụ: Một đối tượng Person có một đối tượng Passport.
- Trong logical design (CSDL quan hệ):
  - Chuyển đổi:
    - Tạo ai bảng Person và Passport và thêm một khóa ngoại một trong hai bảng (thường là bảng "được chứa" trong quan hệ này).
  - Phương án :
    - Thêm một khóa ngoại vào bảng Person trỏ đến bảng Passport.
    - Mỗi người chỉ có một hộ chiếu, vì vậy mỗi dòng trong bảng Person sẽ chứa một khóa ngoại trỏ đến bảng Passport.

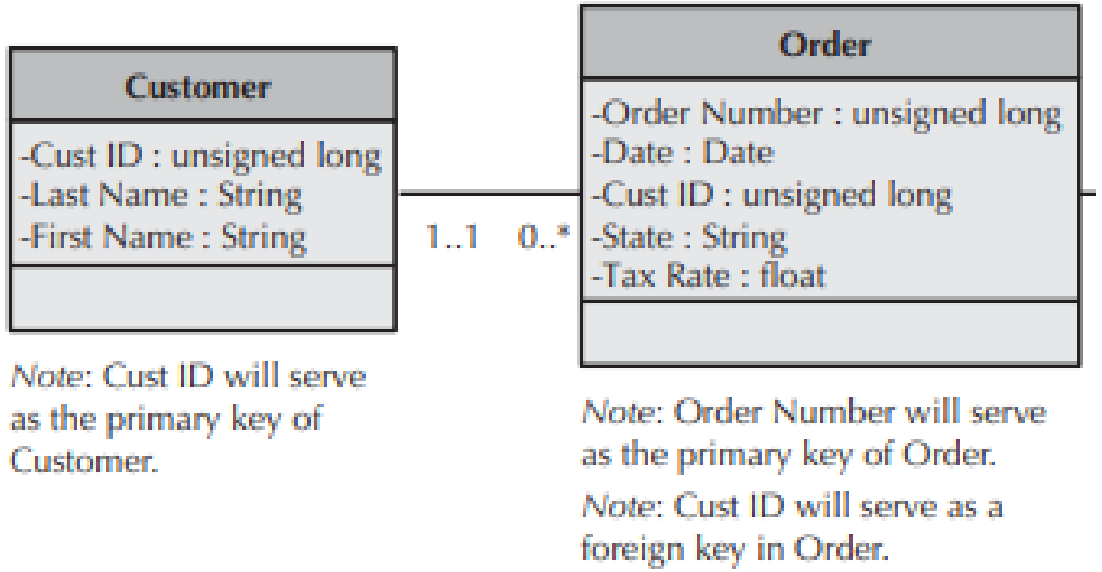
Mối quan hệ 1-n (One-to-Many)

- Trong mô hình OO:
  - Một đối tượng A có thể liên kết với nhiều đối tượng B, nhưng mỗi đối tượng B chỉ liên kết với một đối tượng A.
  - Ví dụ: Một đối tượng Customer có thể đặt nhiều Order, nhưng mỗi Order chỉ thuộc về một Customer.
- Trong logical design (CSDL quan hệ):
  - Chuyển đổi: thêm một khóa ngoại vào bảng Order, trỏ đến bảng Customer.
  - Cấu trúc bảng:
    - Customer: CustomerID (PK), Name, Address
    - Order: OrderID (PK), OrderDate, CustomerID (FK)
  - Mỗi bản ghi trong bảng Order chứa khóa ngoại CustomerID để chỉ ra rằng đơn hàng này thuộc về khách hàng nào.

ví dụ. 1-n

- Sơ đồ lớp cho thấy một CUSTOMER có thể có nhiều ORDER.
- Khi chuyển đổi, khóa chính CustomerID "di chuyển" sang bảng ORDER để tạo liên kết.

- (Sơ đồ UML Class Diagram)
  - CUSTOMER --- 1..1 ------ 0..* --- ORDER
- (Sơ đồ Lược đồ Quan hệ)
  - CUSTOMER (CustomerID, CustomerName, ...)
  - ORDER (OrderID, OrderDate, ..., CustomerID)
  - (Khóa chính được in đậm, khóa ngoại được in nghiêng)



Mối quan hệ n-n (Many-to-Many)

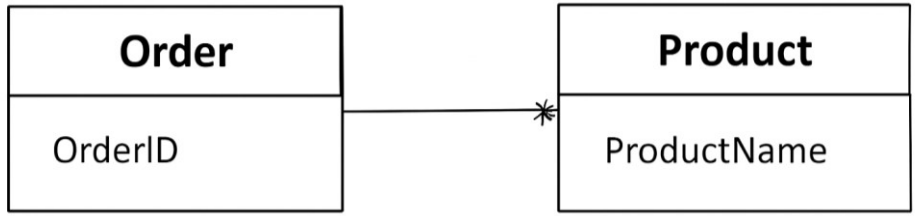
- Trong mô hình OO:
  - Một đối tượng A có thể liên kết với nhiều đối tượng B, và ngược lại,
  - Ví dụ: Một Student có thể tham gia nhiều Course, và một Course có thể có nhiều Student.
- Trong logical design (CSDL quan hệ):
  - Chuyển đổi: Để chuyển mối quan hệ n-n sang cơ sở dữ liệu quan hệ, cần tạo một bảng liên kết (junction table), nơi chứa các khóa ngoại của hai bảng liên quan.
  - Cấu trúc bảng:
    - Student: StudentID (PK), Name
    - Course: CourseID (PK), CourseName
    - StudentCourse: StudentID (FK), CourseID (FK) (bảng liên kết giữa Student và Course).
  - Mỗi dòng trong bảng StudentCourse đại diện cho một mối quan hệ giữa một học sinh và một khóa học.
  - Do đó, mỗi học sinh có thể tham gia nhiều khóa học và mỗi khóa học có thể có nhiều học sinh

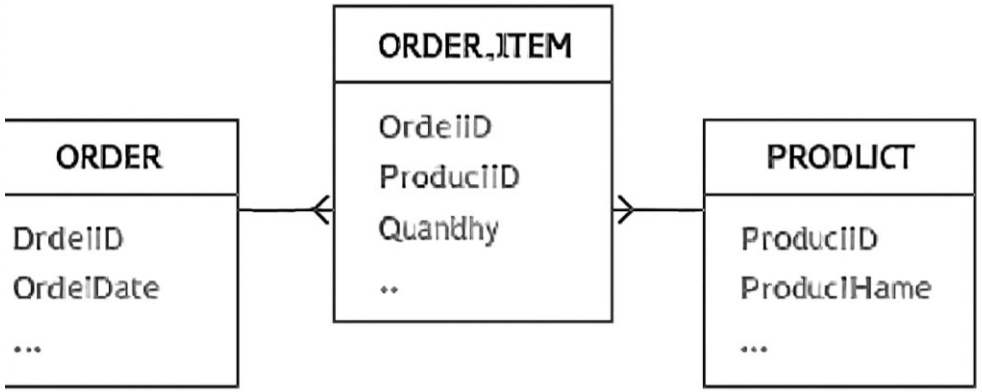
junction table (associative table).

- Bảng cầu nối (associative table).
  - Chuẩn hóa chính là cơ chế buộc chúng ta phải tạo ra một bảng kết hợp
  - Bảng này sẽ chứa khóa ngoại từ hai bảng gốc,.
- Cặp khóa ngoại này thường tạo thành khóa chính phức hợp (composite key) cho bảng mới.
- Lớp kết hợp trong Sơ đồ Lớp sẽ được ánh xạ trực tiếp thành bảng kết hợp này.
- Ví dụ:
  - ORDER (N) và PRODUCT (N)
    - Bảng ORDER có khóa chính OrderID.
    - Bảng PRODUCT có khóa chính ProductID.
    - Tạo bảng mới ORDER_ITEM (OrderID, ProductID, Quantity, ...).

Ví dụ quan hệ nhiều-nhiều giữa ORDER và PRODUCT được giải quyết.

- Sơ đồ lớp cho thấy một đơn hàng có thể có nhiều sản phẩm và một sản phẩm có thể nằm trong nhiều đơn hàng.
- Khi chuyển đổi, một bảng trung gian ORDER_ITEM được tạo ra để lưu trữ chi tiết của từng dòng sản phẩm trong mỗi đơn hàng.
- (Sơ đồ UML Class Diagram)
  - ORDER --- * ------ * --- PRODUCT
- ➔
- (Sơ đồ Lược đồ Quan hệ)
  - ORDER (OrderID, OrderDate, …)
  - ORDER_ITEM (OrderID, ProductID, Quantity, …)
    PRODUCT (ProductID, ProductName, …)





Quy tắc 5: Ánh xạ Kế thừa (Generalization/Inheritance)

- Thách thức: CSDL quan hệ không hỗ trợ kế thừa.
  - Trong OOAD: Kế thừa là một khái niệm tự nhiên và mạnh mẽ.
  - Trong RDBMS: CSDL quan hệ không có khái niệm kế thừa.
- Giải pháp:
  - Chúng ta phải "giả lập" nó bằng các cấu trúc bảng.
  - Quy tắc ánh xạ phải xác định cách biểu diễn hệ thống phân cấp lớp (lớp con kế thừa từ lớp cha) trong cơ sở dữ liệu
  - Các quyết định thiết kế này có liên quan mật thiết đến các nguyên tắc chuẩn hóa.
- Có ba chiến lược phổ biến:
  1. Table per Hierarchy) - Một bảng cho toàn bộ hệ thống phân cấp :
  2. Table per Concrete – Class Bảng theo lớp con
  3. Table per Type – Class Bảng cho mỗi lớp

5.1 Một bảng cho toàn bộ hệ thống phân cấp (Table per Hierarchy)

- Tạo một bảng duy nhất cho toàn bộ phân cấp (hierarchy),
  - Các cột trong bảng sẽ là sự hợp nhất các thuộc tính của các lớp cha và con.
  - Một cột phân biệt để xác định mỗi hàng thuộc về lớp con cụ thể nào.
- Ví dụ:
  - lớp cha Vehicle và các lớp con như Car, Truck, Bike.
  - Cột VehicleType xác định kiểu xe (car, truck, bike), và các cột khác có thể chứa giá trị NULL tùy vào từng lớp con.

ID	VehicleType	Wheels	EngineType	CargoCapacity
1	Car	4	Petrol	NULL
2	Truck	6	Diesel	1000
3	Bike	2	Electric	NULL
0	Vehicle	NULL	NULL	NULL

Đặc điểm

- Truy vấn đa hình và không đa hình nhanh:
  - các truy vấn nhanh, thông tin được lưu trữ trong bảng và được kiểm soát bởi cột,
- Bảng được tạo tối thiểu:
  - chỉ tạo ra một bảng duy nhất — bất kể tính kế thừa phức tạp đến đâu.
- Tiến hóa lược đồ đơn giản:
  - Việc thêm và xóa thuộc tính cũng đơn giản như việc thêm và xóa các cột trong bảng
- Cơ sở dữ liệu không chuẩn hóa:
  - Vi phạm này xảy ra do các thuộc tính phụ thuộc vào khóa, dẫn đến sự phụ thuộc vào một thuộc tính không phải khóa chính
- Cột có thể null:
  - Thuộc tính lớp con tạo ra các cột có thể null trong bảng được tạo ra.

Table-per-Hierarchy (TPH):

- Ưu điểm:
  - Hiệu quả truy vấn: Chỉ cần một bảng duy nhất, giúp truy vấn trở nên đơn giản .
  - Ít bảng hơn: Không phải quản lý nhiều bảng.
- Nhược điểm:
  - Dữ liệu thừa: Nếu các lớp con có nhiều thuộc tính không dùng đến, phải lưu trữ giá trị NULL cho các thuộc tính
  - Khó mở rộng: Khi phân cấp phức tạp hơn, việc quản lý dữ liệu trở nên khó khăn.

5.2. Một bảng cho mỗi lớp cụ thể (Table per Concrete -Class):

- Mỗi lớp con có một bảng riêng biệt, các bảng con chứa tất cả các thuộc tính (cả lớp cha và lớp con).
  - tất cả các dữ liệu sẽ được lưu trữ trong đó, mà không cần phải có bảng cha.
  - điều này loại bỏ mối quan hệ kế thừa giữa các lớp
  - tất cả các thuộc tính, bao gồm cả thuộc tính được kế thừa, đều được ánh xạ vào một cột riêng biệt
- Tuân theo Dạng chuẩn thứ ba (3NF):
  - đảm bảo cơ sở dữ liệu của bạn phù hợp với 3NF, giảm khả năng xảy ra mâu thuẫn về mặt logic.

Table per Concrete -Class

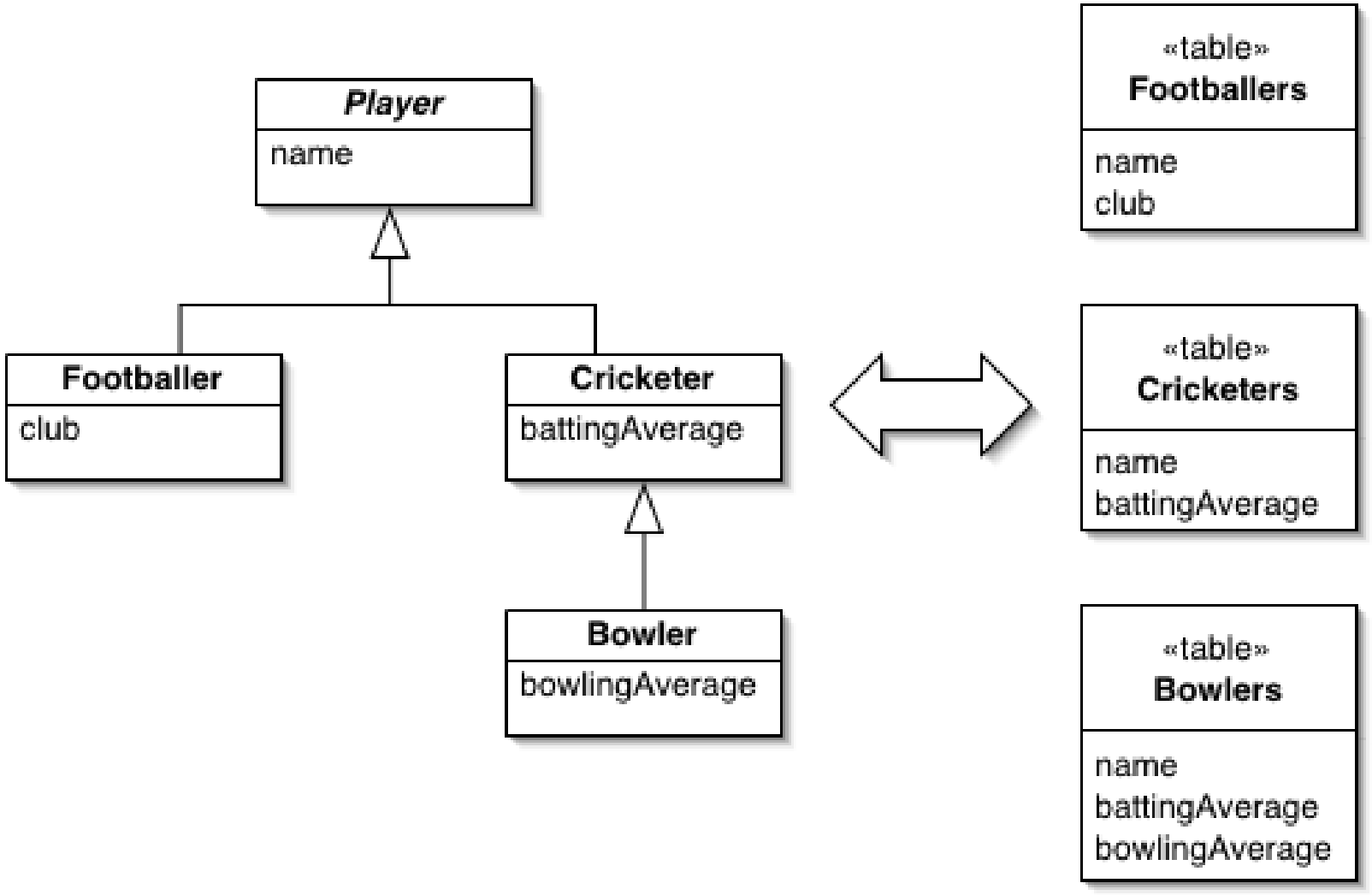


Table-per-Concrete-Class (TPC)

- Ưu điểm:
  - Việc coi mỗi đối tượng như một thực thể riêng biệt trực quan và dễ hiểu cho lập trình viên.
  - Hiệu quả truy vấn: Vì mỗi bảng chứa toàn bộ dữ liệu cho một lớp con, không cần phải thực hiện phép JOIN.
  - Không có dữ liệu thừa: Không cần phải lưu trữ các giá trị NULL, vì mỗi lớp con chứa tất cả thông tin cần thiết.
- Nhược điểm:
  - Dữ liệu trùng lặp: Các thuộc tính của lớp cha sẽ bị trùng lặp trong mỗi bảng con.
  - Khó khăn trong việc duy trì tính toàn vẹn:
    - thay đổi một thuộc tính chung của lớp cha, phải thay đổi tất cả các bảng con.
  - Truy vấn đa hình chậm:
    - mỗi lớp dẫn xuất đều có bảng riêng, truy vấn đa hình lớn và không hiệu quả.

5.3. Một bảng cho mỗi lớp (Table per Type - Class)

- Tạo một bảng cho lớp cha và một bảng riêng cho mỗi lớp con
  - Mỗi lớp, dù trừu tượng hay không, đều sử dụng một bảng riêng ,
  - Các bảng con chỉ lưu trữ các thuộc tính riêng biệt của lớp con, các cột chỉ chứa các thuộc tính không được kế thừa từ lớp cha.
  - bảng cha sẽ chứa các thuộc tính chung cho tất cả các lớp con.
  - Khóa chính của mỗi lớp con là khóa ngoại của bảng lớp cha .
- Tuân theo Dạng chuẩn thứ ba (3NF):
  - Giống như TPC, tuân theo 3NF và đảm bảo rằng cơ sở dữ liệu được chuẩn hóa.

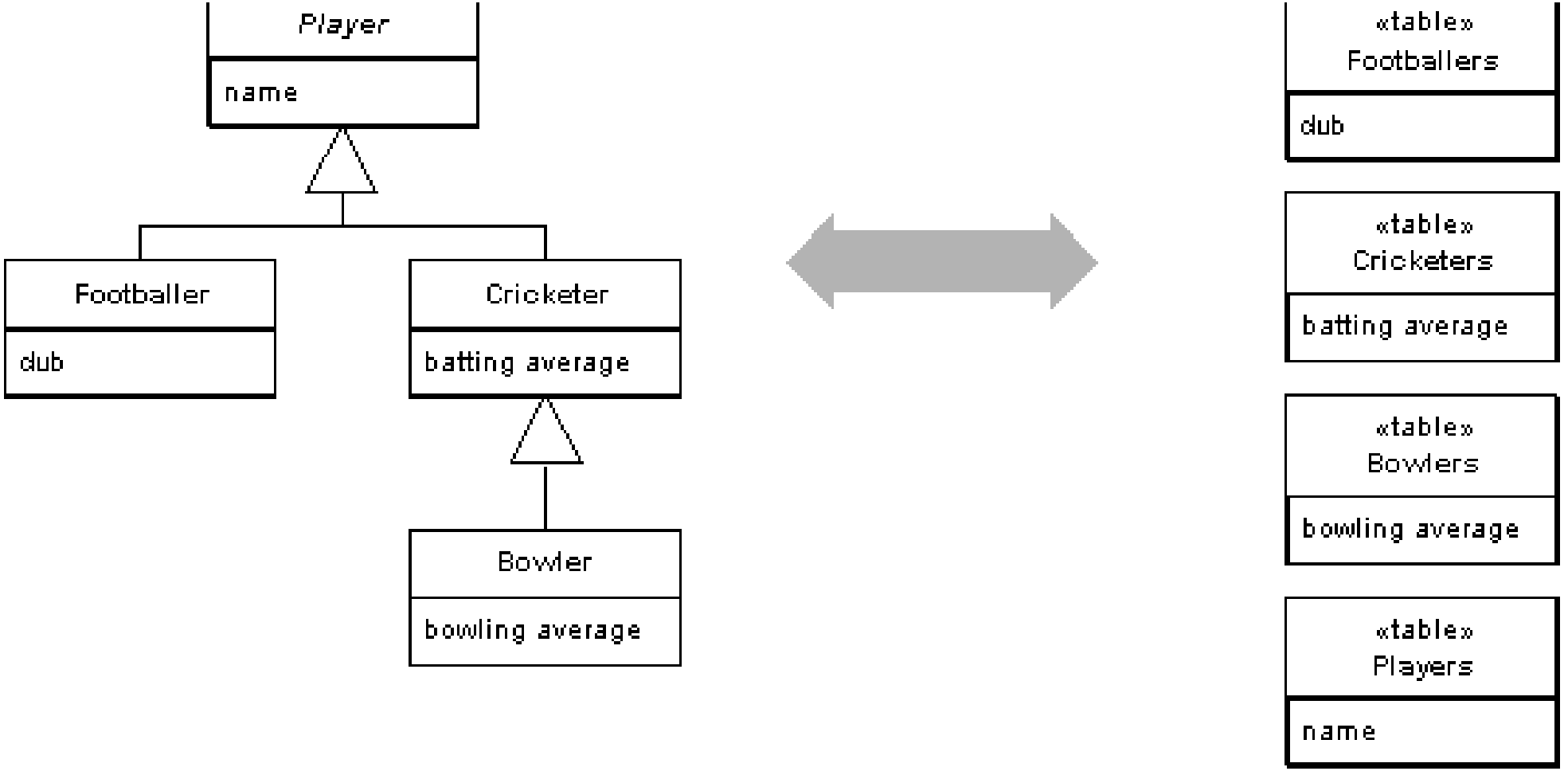

Player
name
Footballer
club
Cricketer
batting average
Bowler
bowling average
«table»
Footballers
club
«table»
Cricketers
batting average
«table»
Bowlers
bowling average
«table»
Players
name

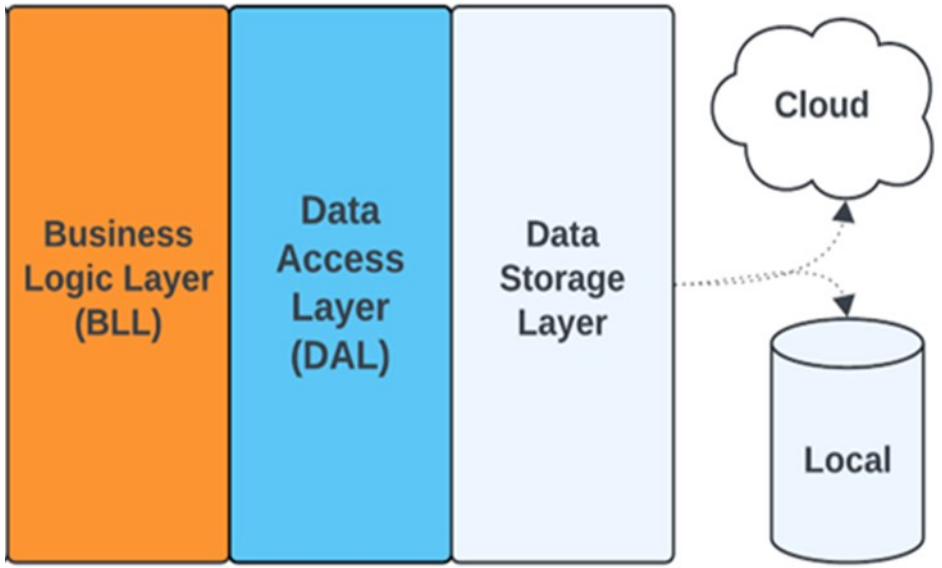
Table-per-Subclass (TPS):

- Ưu điểm:
  - Không có dữ liệu thừa: Mỗi bảng chỉ chứa các thuộc tính liên quan đến lớp đó, tránh việc lưu trữ các giá trị NULL.
  - Quá trình tiến hóa mô hình rất đơn giản: thay đổi, thêm và xóa các lớp khỏi hệ thống phân cấp trở nên dễ dàng chỉ ảnh hưởng đến bảng tương ứng,
  - Dễ dàng mở rộng: Khi thêm lớp mới vào phân cấp, chỉ cần tạo một bảng mới cho lớp đó.
- Nhược điểm:
  - Số lượng lớn bảng được tạo: yêu cầu một bảng tương ứng cho tất cả các lớp nên số lượng bảng được tạo sẽ lớn
  - Hiệu suất truy vấn chậm: Các câu lệnh SQL được tạo từ các truy vấn phức tạp tùy thuộc vào số lượng cấp trong hệ thống phân cấp lớp.
  - Trong các hệ thống phân cấp có nhiều cấp, vấn đề này càng trở nên trầm trọng hơn.

Data Storage

Data Management Layer

- Để đạt được sự bền vững cho đối tượng dữ liệu,
  - chúng ta cần một cơ chế chuyên biệt.
  - Trong kiến trúc phần mềm, thuộc về tầng riêng biệt gọi là DML
- Data Management Layer
  - Chịu trách nhiệm về việc các đối tượng sẽ được lưu trữ và truy xuất như thế nào.
  - giúp tách biệt logic nghiệp vụ khỏi các chi tiết kỹ thuật về lưu trữ.
- DML đóng vai trò như một cầu nối giữa logic nghiệp vụ và phương tiện lưu trữ vật lý.



Data Architecture

- Data Architecture là cấu trúc và cách thức tổ chức thu thập, lưu trữ và quản lý dữ liệu
  - kiến trúc dữ liệu cung cấp bản thiết kế và cấu trúc tổng thể,
  - quản lý dữ liệu đề cập đến việc thực thi các chính sách và quy trình nhằm duy trì cấu trúc đó trong suốt vòng đời của dữ liệu.
- Thiết kế Cơ sở dữ liệu là một hoạt động trong Tầng Quản lý Dữ liệu (Data Management Layer).
  - 1. Data Storage Lưu trữ Dữ liệu
    - Quản lý cách dữ liệu được lưu trữ và xử lý bởi chương trình.
  - 2. Data Access Logic Logic Truy cập Dữ liệu
    - Xử lý các yêu cầu truy cập dữ liệu, thường là các truy vấn cơ sở dữ liệu
    - (ví dụ: SQL).

Data Storage Hệ thống lưu trữ dữ liệu

- Data Storage
  - xác định cách thức dữ liệu được tổ chức, quản lý và lưu trữ vật lý trong hệ thống cơ sở dữ liệu để đảm bảo hiệu quả, tính toàn vẹn và khả năng truy cập.
- DS đề cập đến các phương pháp và công nghệ lưu trữ gồm
  - việc lựa chọn các mô hình phù hợp,
  - xác định bố cục vật lý và
  - tối ưu hóa hiệu suất để đáp ứng
- Vi dụlưu trữ dữ liệu kinh doanh như thế nào?
  - Để đảm bảo tính khả dụng, bảo mật và hiệu suất của dữ liệu,
  - Việc lựa chọn công nghệ lưu trữ phụ thuộc vào
    - loại,
    - khối lượng và
    - cách sử dụng dữ liệu,

Các phương pháp lưu trữ dữ liệu

- 1. Network Attached Storage (NAS)
  - NAS là một thiết bị lưu trữ dữ liệu duy nhất mà nhiều máy có thể chia sẻ qua một mạng.
  - Lợi ích lưu trữ tập trung và khả năng cộng tác, chi phí thấp hơn và thiết lập đơn giản tuy nhiên tốc độ hạn chế
- 2. Direct Attached Storage (DAS)
  - Ổ cứng DAS(usb, ssd) được kết nối trực tiếp với máy tính
  - Nhanh khả năng chia sẻ kém
- 3. Cloud Storage
  - Lưu trữ đám mây sử dụng internet
  - mở rộng dễ,chi phí thiết lập và vận hành thấp
- 4. Storage Area Network (SAN)
  - lưu trữ dựa trên mạng kết hợp nhiều thiết bị và loại lưu trữ.
  - hiệu suất nhanh và lý tưởng cho nhiều người dùng cung cấp khả năng mở rộng không giới hạn.

Các giải pháp và loại lưu trữ dữ liệu phổ biến

- Data Warehouse :
  - trữ tập trung để lưu trữ dữ liệu có cấu trúc từ nhiều nguồn,
  - được tối ưu hóa cho (BI), báo cáo và phân tích.
- Data lake:
  - Được thiết kế để lưu trữ lượng lớn dữ liệu thô, đa cấu trúc (hình ảnh, video, tài liệu) ở định dạng gốc.
  - lý tưởng cho các dự án xử lý dữ liệu lớn, phân tích thời gian thực và học máy.
- Data Lakehouse :
  - Kiến trúc quản lý dữ liệu mở, hiện đại
  - kết hợp tính linh hoạt và khả năng mở rộng của hồ dữ liệu với khả năng quản lý và giao dịch của kho dữ liệu.

Data Warehouse

- Kho dữ liệu
  - là phương pháp lưu trữ phổ biến đối với các doanh nghiệp quy mô vừa để quản lý dữ liệu theo cách linh hoạt (agile).
  - dữ liệu lưu trữ có cấu trúc, có thể chuyển ngay đến công cụ BI để phân tích và báo cáo.
- Ưu điểm:
  - Dữ liệu tập trung, Cải thiện chất lượng dữ liệu,
    - Tạo điều kiện thuận lợi cho sự hợp tác phòng ban
  - Tăng hiệu quả, Khả năng mở rộng và hiệu năng
    - có thể thu thập và lưu trữ dữ liệu hoàn toàn tự động thông qua tích hợp dữ liệu.
  - Phân tích nâng cao,Tối ưu hóa cho truy vấn.
- Nhược điểm:
  - Chi phí cao và độ phức tạp lớn:
    - Tốn kém trongtriển khai, bảo trì và đòi hỏi kỹ năng chuyên môn.
  - Quá trình thiết lập tốn nhiều thời gian, Thách thức trong việc tích hợp dữ liệu
  - Không lưu trữ dữ liệu thô, cần quá trình ETL (Extract, Transform, Load).

Kho dữ liệu so với cơ sở dữ liệu

- Cơ sở dữ liệu (OLTP-Optimized for fast, frequent *transactions*):
  - Được tối ưu hóa cho các giao dịch nhanh chóng, thường xuyên (thêm, cập nhật) phục vụ các hoạt động hàng ngày (ví dụ: xử lý đơn đặt hàng).
- Kho dữ liệu (OLAP-Optimized for complex *analytical queries*):
  - Được tối ưu hóa cho các truy vấn phân tích phức tạp trên các tập dữ liệu lớn để tìm ra thông tin chi tiết
  - (ví dụ: phân tích xu hướng bán hàng trong năm năm).

- Kho dữ liệu con (data mart)
  - phiên bản thu nhỏ của kho dữ liệu chính (data warehouse),
  - được thiết kế để đáp ứng nhu cầu thông tin của một nhóm nhỏ người dùng cuối đồng nhất,
    - Vd một phòng ban hoặc đơn vị kinh doanh.

Data Lake

- Hồ dữ liệu
  - Là chiến lược lưu trữ dữ liệu giúp tổ chức lưu trữ một lượng lớn dữ liệu thường ở định dạng thô tự nhiên không yêu cầu cấu trúc hoặc lược đồ được xác định trước khi lưu.
- Ưu điểm:
  - Linh hoạt và có thể lưu trữ bất kỳ loại dữ liệu nào.
  - Chi phí thấp: Tận dụng các dịch vụ lưu trữ đối tượng giá rẻ .
  - Khả năng mở rộng mạnh mẽ:
  - Hỗ trợ phân tích chuyên sâu:
    - Machine Learning , AI và phân tích dự báo cận dữ liệu gốc
- Nhược điểm:
  - Khó khăn trong việc quản lý và truy vấn dữ liệu.
    - Nguy cơ trở thành "Đầm lầy rác dữ liệu" (Data Swamp): .
  - Độ phức tạp cao:
    - Yêu cầu chuyên gia (Data Engineer, Data Scientist) xử lý và khai thác dữ liệu thô.
  - Vấn đề bảo mật và tuân thủ:
  - Hiệu năng truy vấn SQL thấp:

Data Lakehouse

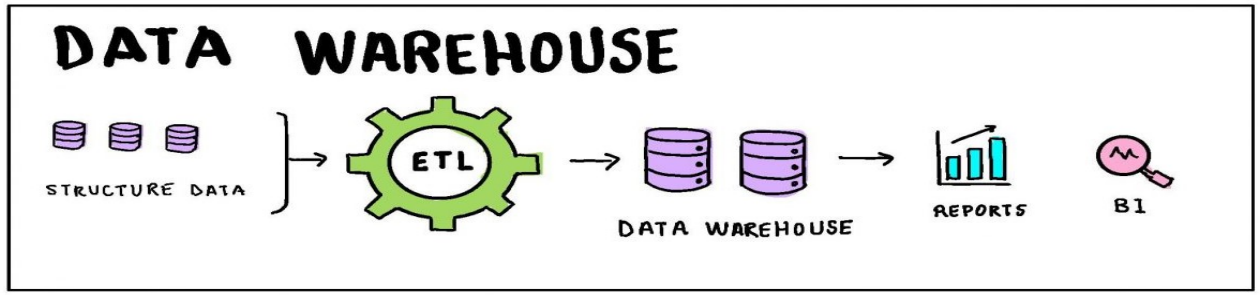
- Data Lakehouse
  - là một kiến trúc quản lý dữ liệu hiện đại kết hợp các đặc tính tốt nhất của Data Lake (tính linh hoạt, chi phí thấp) và Data Warehouse (quản lý cấu trúc, hiệu suất truy vấn cao).
  - 2025, Data Lakehouse đang trở thành tiêu chuẩn cho các doanh nghiệp triển khai AI và phân tích dữ liệu trên một nền tảng thống nhất.
- Ưu điểm của Data Lakehouse
  - Hỗ trợ đa dạng loại dữ liệu: tương tự như Data Lake.
  - Tiết kiệm chi phí:
  - Tính toàn vẹn dữ liệu (ACID): Đảm bảo các giao dịch dữ liệu an toàn, tin cậy thông qua việc hỗ trợ tính chất ACID,
  - Hiệu suất phân tích cao: Cho phép các công cụ BI (như Power BI, Tableau) truy vấn trực tiếp với tốc độ nhanh nhờ các lớp quản lý schema và tối ưu hóa index.
  - Phục vụ tốt cho AI/ML: có thể truy cập dữ liệu thô trực tiếp cho các mô hình Machine Learning mà không cần qua quy trình ETL phức tạp.

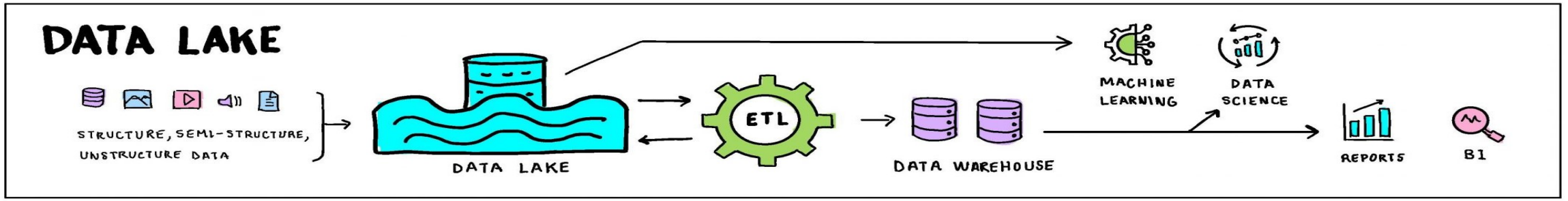
Nhược điểm và thách thức

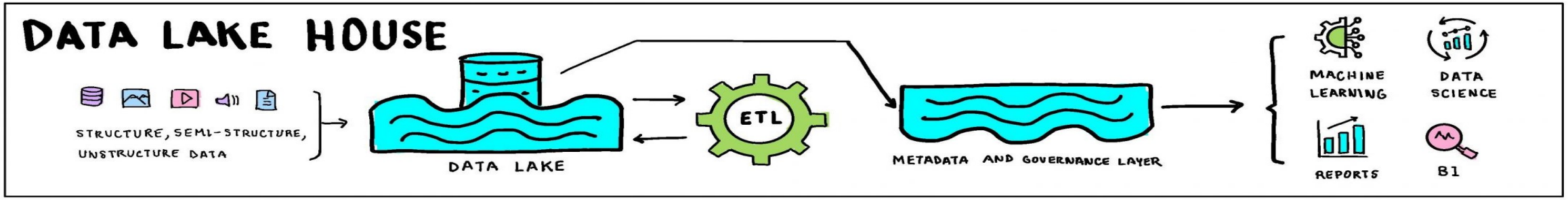
- Độ phức tạp trong triển khai:
  - Việc xây dựng một hệ thống Data Lakehouse từ đầu đòi hỏi kỹ năng chuyên môn cao về kiến trúc dữ liệu hiện đại.
- Quản trị dữ liệu (Governance):
  - việc áp dụng các chính sách bảo mật và kiểm soát chất lượng đồng nhất cho toàn bộ hệ thống trở nên khó khăn hơn.
- Sự trưởng thành của công nghệ:
  - các công cụ và hệ sinh thái cho Lakehouse vẫn đang trong quá trình hoàn thiện và phát triển nhanh chóng.

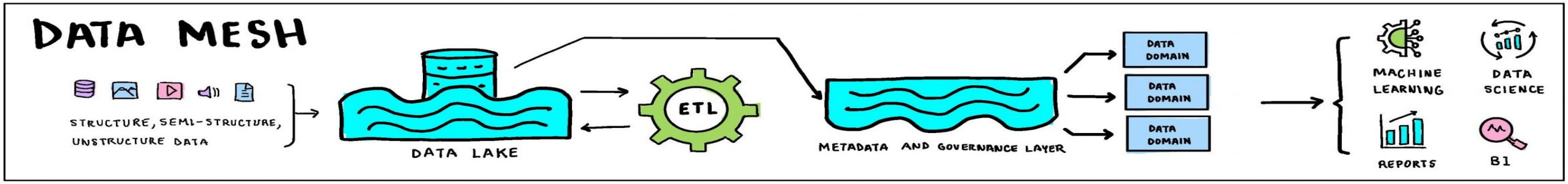
So sánh

CC by SA 2.0









Data Mesh

- (Lưới dữ liệu)
  - là một mô hình kiến trúc và tổ chức dữ liệu phi tập trung, trong đó dữ liệu được quản lý theo từng lĩnh vực nghiệp vụ cụ thể (domain) thay vì tập trung vào một kho dữ liệu duy nhất.
- Ưu điểm
  - Khả năng mở rộng cao (Scalability):
    - không gặp tình trạng "nút thắt cổ chai" tại đội ngũ IT trung tâm.
  - Chất lượng dữ liệu tốt hơn:
    - Các chuyên gia trong từng lĩnh vực (domain experts).
  - Tốc độ đổi mới nhanh:
    - Các nhóm có thể tự do phát triển và triển khai mô hình AI/ML
  - Tính linh hoạt:
    - cho phép các bộ phận sử dụng các công nghệ và công cụ phù hợp nhất với nhu cầu cụ thể của họ thay vì phải tuân theo một tiêu chuẩn gò bó duy nhất.

Nhược điểm

- Độ phức tạp trong quản lý:
  - hệ thống phi tập trung yêu cầu khả năng quản trị liên bang (federated governance) mạnh tránh dữ liệu rời rạc và không đồng nhất.
- Chi phí ban đầu cao:
  - Đòi hỏi đầu tư lớn vào việc tái cấu trúc tổ chức, đào tạo nhân sự và tích hợp nhiều loại công nghệ khác nhau do chưa có giải pháp "tất cả trong một" từ các nhà cung cấp.
- Thách thức về văn hóa:
  - Chuyển đổi sang Data Mesh đòi hỏi sự thay đổi tư duy từ "sở hữu dữ liệu để lưu trữ" sang "sở hữu dữ liệu để phục vụ",
- Trùng lặp nỗ lực:
  - các nhóm khác nhau có thể thực hiện những công việc giống nhau (như cùng xây dựng một bộ lọc dữ liệu khách hàng), dẫn đến lãng phí tài nguyên.

Vai trò của Data Storage

1. Nền tảng:
   - Là tầng thấp nhất, nơi dữ liệu được ghi nhận và giữ lại sau khi được thu thập.
2. Quản lý vòng đời:
   - Xác định cách dữ liệu được lưu trữ, truy xuất và quản lý trong suốt vòng đời của dữ liệu.
3. Hỗ trợ các tầng khác:
   - Cung cấp dữ liệu thô (raw data) hoặc đã xử lý cho các tầng trên như
     - xử lý dữ liệu (processing), biến đổi (transformation) và truy cập/phân tích (BI/Analytics).
4. Đảm bảo tính nhất quán & bảo mật:
   - Giúp thực thi các quy tắc và tiêu chuẩn về dữ liệu,
   - đảm bảo chất lượng và quyền truy cập.

Các tiêu chuẩn lựa chọn công nghệ lưu trữ

- Tính ACID (Atomicity, Consistency, Isolation, Durability) hay Acid
  - tập các thuộc tính đảm bảo tính toàn vẹn dữ liệu ngay cả khi xảy ra lỗi hoặc sự cố
  - Điều này rất quan trọng đối với các hệ thống cần độ chính xác dữ liệu cao,(ngân hàng),
- Base(Basically Available, Soft state, Eventual consistency):
  - Base ưu tiên tính linh hoạt, tính khả dụng hơn tính nhất quán tức thời
- Định lý CAP:
  - Sự cân bằng giữa tính nhất quán (Consistency),
  - tính sẵn sàng (Availability) và
  - khả năng chịu lỗi phân đoạn (Partition Tolerance).
- Data Storage cần phải được thiết kế phù hợp với chiến lược tổng thể của Enterprise Architecture:
    - Khả năng mở rộng (Scalability): Cho phép hệ thống lớn mạnh mà không cần thiết kế lại toàn bộ.

Tổng kết bài học

- Vai trò của CSDL:
  - Cung cấp cơ chế lưu trữ bền vững (object persistence), là nền tảng của hầu hết các ứng dụng.
- Chuyển đổi OOAD → RDB:
  - Một quy trình có hệ thống để ánh xạ các lớp, thuộc tính và mối quan hệ thành các bảng, cột và khóa ngoại.
- Các mức thiết kế:
  - Từ Khái niệm (Conceptual) → Logic → Vật lý (Physical).
- Chuẩn hóa:
  - Là kỹ thuật cần thiết để đảm bảo tính toàn vẹn dữ liệu, loại bỏ dư thừa và dị thường, tạo ra một thiết kế CSDL chất lượng cao.

HUST

hust.edu.vn fb.com/dhbkhn

Bài tập thực hành: Chuẩn hóa Sơ đồ Lớp

- Tình huống:
  - Lớp Enrollment được thiết kế để lưu trữ thông tin đăng ký khóa học của sinh viên.
- Mô hình ban đầu (chưa chuẩn hóa):
- Yêu cầu (theo nhóm):
  - 1. Xác định các vi phạm Dạng chuẩn (1NF, 2NF, 3NF) trong mô hình trên.
  - 2. Tái cấu trúc thành một Sơ đồ Lớp đã được chuẩn hóa.

Gợi ý giải bài tập

- Vi phạm 1NF:
  - (courseID, courseName) là một nhóm lặp.
  - Điều này cho thấy Course nên là một lớp riêng.
- Vi phạm 3NF:
  - studentName phụ thuộc bắc cầu vào enrollmentID (thông qua studentID). studentName thuộc về lớp Student.
- Vi phạm 3NF:
  - courseName phụ thuộc bắc cầu vào enrollmentID (thông qua courseID). courseName thuộc về lớp Course.

Bài tập trên lớp: Hệ thống Quản lý Thư viện

- Chúng ta có một sơ đồ lớp đơn giản cho hệ thống quản lý thư viện.
- Nhiệm vụ của chúng ta là chuyển đổi nó thành một lược đồ quan hệ hoàn chỉnh.

- (Hiển thị Sơ đồ Lớp)
  - BORROWER (BorrowerID, Name) --- 1..1 ------ 0..* --- LOAN (LoanID, DueDate) --- 1..1 ------ 0..* --- BOOK (BookID, Title)
- Yêu cầu:
  - 1. Xác định các bảng cần tạo.
  - 2. Xác định khóa chính cho mỗi bảng.
  - 3. Xác định các mối quan hệ (1-N, N-N).
  - 4. Thêm các khóa ngoại cần thiết.
  - 5. Viết ra lược đồ quan hệ cuối cùng.

Giải pháp bài tập trên lớp

- Chúng ta có ba bảng tương ứng với ba lớp.
- Mối quan hệ giữa BORROWER và LOAN là 1-N, và giữa BOOK và LOAN cũng là 1-N.
- Do đó, khóa chính từ BORROWER và BOOK được thêm làm khóa ngoại vào bảng LOAN.
- Nội dung:
- Bước 1&2: Các bảng và khóa chính:
  - BORROWER (BorrowerID, Name, ...)
  - LOAN (LoanID, DueDate, ...)
- BOOK (BookID, Title, ...)
  - Bước 3&4: Quan hệ và khóa ngoại:
  - Một BORROWER có nhiều LOAN (1-N) → Thêm BorrowerID vào bảng LOAN.
  - Một BOOK có nhiều LOAN (1-N) → Thêm BookID vào bảng LOAN.
- Bước 5: Lược đồ quan hệ cuối cùng:
  - BORROWER (BorrowerID, Name, ...)
  - BOOK (BookID, Title, ...)
  - LOAN (LoanID, DueDate, ..., BorrowerID, BookID)

Vấn đề của Dữ liệu chưa Chuẩn hóa UNF trong Mô hình Lớp

- Dư thừa dữ liệu (Data Redundancy):
  - Cùng một thông tin được lưu trữ ở nhiều nơi.
  - Ví dụ: Tên và địa chỉ khách hàng bị lặp lại trong mỗi đối tượng Order.
- Dị thường Cập nhật (Update Anomaly):
  - Khi thay đổi một thông tin, phải thay đổi ở tất cả các nơi nó xuất hiện.
  - Ví dụ: Nếu khách hàng đổi địa chỉ, phải cập nhật tất cả các đối tượng Order của họ.
- Dị thường Chèn (Insertion Anomaly):
  - Không thể thêm một đối tượng mới nếu thiếu thông tin từ đối tượng khác.
  - Ví dụ: Không thể thêm một sản phẩm mới vào hệ thống nếu nó chưa từng được đặt hàng.
- Dị thường Xóa (Deletion Anomaly):
  - Xóa một đối tượng có thể vô tình làm mất thông tin quan trọng không liên quan.